Inventory Intelligence System

Granular Developer Reference — Full SRS v3.0

Multi-Tenant SaaS Platform for Automobile Parts Retailers

Prepared By	Solution Architect / Consultant
Intended For	Development Team / AI Agent
Document Version	v2.0 — Full Granular Dev Reference
Classification	Internal — NOT for Client Distribution
Date	March 2025

PURPOSE: This document is self-contained. A developer or agentic AI reading this document should be able to build the complete IIS platform without any additional input.

Table of Contents

Table of Contents....1
1. Introduction & Business Context....6
1.1 What is IIS?....6
1.2 Pilot Tenant Profile....6
1.3 How to Read This Document....6
2. System Architecture....6
2.1 Architecture Pattern....7
2.2 Recommended Tech Stack....7
2.3 Multi-Tenancy Model....7
2.4 User Roles & Access Matrix....7
2.5 URL Structure....8

3. Data Models 8
3.1 Shared Schema (public) 8
3.1.1 tenants 8
3.1.2 plans 9
3.2 Tenant Schema Tables 10
3.2.1 skus (SKU Master) 10
3.2.2 sales 10
3.2.3 purchases 11
3.2.4 inventory_snapshots 11
3.2.5 customers 12
3.2.6 sales_invoices 12
3.2.7 payment_receipts 13
3.2.8 outstanding_ledger 13
3.2.7 import_batches 14
3.2.8 cost_decode_formulas 15
3.2.9 whatsapp_templates 15
3.2.10 forecasting_cache 15
3.2.11 branches 16
3.2.12 branch_column_maps 17
3.2.12 stock_transfers 17
4. Module: Data Import & Management 18
4.1 User Stories 18
Urgent SKU Handling in PO Advisor — Confirmed Decision 19
4.2 Pages 19
4.3 Import Processing Logic 22
4.3.1 Column Mapping 22
4.3.2 Master Auto-Creation Rules 22
4.3.3 Sale From / Location Auto-Detection & Branch Auto-Creation 23
4b. Module: Branch & Location Management 24
4b.1 User Stories 24
4b.2 Pages 26
4b.3 Branch Context in All Modules 30
4b.4 Effective Stock Calculation with Transfers 31
4A. Busy Web Service Integration 31
4A.1 Busy Service Codes Reference 32
4A.2 Busy VchType and MasterType Constants 32
4A.3 Data Sync Design — Full Sync and Delta Sync 33
4A.3.1 Phase 1 — Initial Full Sync 33
4A.3.2 Phase 2 — Incremental Delta Sync 33

4A.3.3 sync_log Table 34
4A.3.4 Upsert Strategy — No Duplicate Data....34
4A.4 Busy SQL Queries for IIS Data 35
4A.4.1 Item Master (SKUs)....35
4A.4.2 Account Master (Customers and Vendors) 35
4A.4.3 Godowns / Warehouses 35
4A.4.4 Sales (VchType = 9)....35
4A.4.5 Purchases (VchType = 2)....36
4A.4.6 Sale Returns (VchType = 3) and Purchase Returns (VchType = 10) 36
4A.4.7 Receipts — Customer Payments (VchType = 14) 36
4A.4.8 Physical Stock (VchType = 61) 36
4A.4.9 Stock Transfers (VchType = 5) 36
4A.5 Python Integration Layer....37
4A.5.1 Core HTTP Client 37
4A.5.2 Sync Scheduler 38
4A.5.3 Error Handling and Retry....38
4A.6 Tenant Configuration — Busy Connection Settings....38
4A.7 Manual Upload as Fallback....39
4A.8 Integration Architecture & Known Limitations 39
4A.8.1 How the Integration Works 39
4A.8.2 Known Limitations 40
4A.8.3 Deployment Checklist....40
5. Module: SKU Master & Season Tagging 40
5.1 User Stories....40
5.2 Pages 41
6. Module: Demand Forecasting Engine....44
6.1 User Stories....44
6.2 Business Logic — All Formulas....44
6.2.1 DRR Calculation 44
6.2.2 Weeks of Inventory (WOI) 44
6.2.3 System-Suggested MSL 45
6.2.4 12-Week Inventory Target 45
6.2.5 Suggested Order Quantity 45
6.2.6 Seasonal Demand Forecasting — 5-Step Engine 45
6.3 Forecasting Computation Trigger....46
7. Module: Inventory Analytics & MSL Review 46
7.1 User Stories....46
7.2 Pages 47
7.3 MSL Override Logic....48

7.4 Settings — Inventory Targets 48
7B. Module: Smart Reorder Screen 49
7B.1 MSL Rules (Updated) 49
7B.2 Two Analysis Buckets 49
7B.2.1 Bucket 1 — Last 7 Days Sold SKUs (Priority Analysis) 49
7B.2.2 Bucket 2 — All SKUs (Broad Alert Scan) 50
7B.3 Smart Reorder Page 50
7B.3.1 Page Header 50
7B.3.2 Table Columns (both buckets) 50
7B.3.3 MSL Inline Edit 51
7B.3.4 MSL Suggestion Prompt 51
7B.4 Reorder Status Flow 51
7B.5 skus_reorder_orders Table (New) 52
7B.6 API Endpoints — Smart Reorder 52
7B.7 User Stories — Smart Reorder Screen 52
7B.8 Acceptance Criteria — Smart Reorder Screen 53
8. Module: Purchase Order Advisor 53
8.1 User Stories 53
8.2 Pages 54
9. Module: Seasonal Planning 55
9.1 User Stories 55
9.2 Pages 56
10. Module: Cost Decoder & Profitability Analytics 57
10.1 User Stories 57
10.2 Cost Decoding Logic 58
10.2.1 Encoding Scheme 58
10.2.2 Decoding Algorithm 58
10.3 Pages 58
11. Module: Customer Outstanding Report 60
11.1 User Stories 60
11.2 Method A — Direct Upload (outstanding_method = 'direct_upload') 61
11.2.1 Data Source 61
11.2.2 Outstanding Computation 61
11.3 Method B — Computed (outstanding_method = 'computed') 61
11.3.1 Data Source 62
11.3.2 Invoice-Level Outstanding 62
11.3.3 Ageing Bucket Assignment 62
11.3.4 Customer Summary 62
11.4 Pages 62

11.4.2 outstanding_followups Table (New) 62
11.4.3 Follow-up Status Values 63
11.4.4 Enhanced Outstanding Report Page 63
11.4.5 Auto-Close Logic 63
11.4.6 Missed Payment Alert Logic 64
11.4.7 Acceptance Criteria — Follow-up 64
12. Module: WhatsApp Integration 65
12.1 User Stories 65
12.2 Technical Implementation 66
12.2.1 WhatsApp URL Scheme 66
12.2.2 Phone Number Formatting 66
12.2.3 Template Variable Substitution 66
12.3 Pages 66
13. Module: Tenant Dashboard 67
13.1 User Stories 67
13.2 Page Spec 67
13.3 Widget Specifications 68
14. Super Admin Portal 69
14.1 User Stories 69
14.2 Pages 70
15. Reports Reference 74
16. API Contracts (Key Endpoints) 75
16.1 Authentication 75
16.2 Data Import 76
16.3 SKU & Forecasting 76
16.4 Reports 76
16.6 Branches 77
16.5 Super Admin 77
17. Non-Functional Requirements 78
18. Acceptance Criteria 79
18.1 Data Import 79
18.2 Forecasting & Inventory 79
18.3 Seasonal Forecasting 79
18.4 Profitability 79
18.5 Customer Outstanding 80
18.6 Super Admin 80
18.8 Branch & Location Management 80

1. Introduction & Business Context

1.1 What is IIS?

The Inventory Intelligence System (IIS) is a multi-tenant SaaS web application built for automobile parts retailers. It ingests data exports from Busy Accounting Software and provides demand forecasting, inventory analytics, purchase order recommendations, profitability analytics, seasonal planning, and customer outstanding management.

The pilot / reference tenant is an automobile parts retailer (Tenant ID: T001). The system is designed to scale to 100+ tenants on shared infrastructure with complete data isolation.

1.2 Pilot Tenant Profile

Attribute	Detail
Business	Automobile parts retail
Vendors	Multiple vendors supported; 3–4 month lead time per vendor
Branches	Two locations: Store (retail front) and Warehouse (stock holding). Both are tagged as 'Sale From' in Busy exports. Branches are auto-created on first import.
Current ERP	Busy Accounting Software (Web Service API on port 981 — see Section 4A; manual CSV/Excel upload retained as fallback)
Historical Data	~2 years of sales, purchase, and stock data
Active SKUs	~2,500–3,000 SKUs
Users	1 Tenant Admin (owner) + 2–3 staff users
Key Pain Points	Stock-outs despite MSL settings; no demand forecasting; urgent SKU tracking via Excel; no profitability visibility

1.3 How to Read This Document

Each module section follows this structure:

1. User Stories — As a [role], I want [feature], so that [value].
2. Page Specifications — Each screen with URL, access roles, all fields and their types/validations.
3. Business Logic — Exact formulas, rules, and calculations the system must implement.
4. Data Requirements — What data must exist / be stored for each feature.

> *Anything marked DEV NOTE is an implementation guidance comment for developers.*

2. System Architecture

2.1 Architecture Pattern

IIS follows a multi-tenant SaaS architecture with schema-per-tenant isolation. All tenants run on shared application infrastructure but each tenant's data lives in a dedicated PostgreSQL schema (e.g., schema name: tenant_t001, tenant_t002).

2.2 Recommended Tech Stack

Layer	Technology	Notes
Frontend	React.js (Next.js)	SSR for dashboard pages; CSR for interactive reports
Backend / API	Node.js + Express or FastAPI (Python)	REST API; Python preferred if ML forecasting is used
Database	PostgreSQL (schema-per-tenant)	Schema: tenant_{id} for full isolation
File Storage	AWS S3 / Cloudflare R2	Uploaded Busy export files stored per tenant
Auth	JWT + Role-Based Access Control	Roles: super_admin, tenant_admin, tenant_user
Background Jobs	BullMQ / Celery	File parsing, forecasting computation, backup jobs
Hosting	VPS (DigitalOcean / AWS EC2)	Single server sufficient for Phase 1 (up to 20 tenants)
Reverse Proxy	Nginx	SSL termination, routing

2.3 Multi-Tenancy Model

Schema-per-tenant approach:

- Each tenant gets a dedicated PostgreSQL schema created at provisioning time.
- All tenant-specific tables (skus, sales, purchases, inventory, customers, etc.) live inside their schema.
- A shared public schema holds: tenants, plans, super_admin_users, audit_log.
- Application layer always prefixes queries with SET search_path TO tenant_{id}; — this must be enforced in middleware.
- No cross-tenant query is ever permitted. Middleware validates that the JWT tenant_id matches the schema being accessed.

> *DEV NOTE: Use a database middleware layer that automatically sets the search_path based on the authenticated tenant's ID extracted from the JWT. This is non-negotiable for data isolation.*

2.4 User Roles & Access Matrix

Capability	Super Admin	Tenant Admin	Tenant User	Unauthenticated
Super Admin Portal	Yes	No	No	No

Manage Tenants & Plans	Yes	No	No	No
Cost Decoding Config	Yes	Yes (own tenant)	No	No
View Decoded Costs / PnL	Yes	Yes (own tenant)	No	No
Upload Busy Data	Yes (any tenant)	Yes	Yes	No
View All Reports	Yes (any tenant)	Yes	Yes	No
Manage Focus SKUs / Season Tags	Yes	Yes	Yes	No
Manage Users (within tenant)	Yes	Yes	No	No

2.5 URL Structure

Two completely separate applications / sub-domains:

- Tenant Portal: app.iis.in/{tenant-slug}/... (or subdomain: {tenant-slug}.iis.in)
- Super Admin Portal: admin.iis.in/... (completely separate, no shared routes)

> *DEV NOTE: The Super Admin portal should be deployable as a separate Next.js app or at minimum a completely separate route namespace with its own auth guard. No tenant user should ever be able to reach any /admin/* route.*

3. Data Models

The following tables define the core entities. All tables in a tenant schema are prefixed with the schema name at query time. Column types use PostgreSQL conventions.

3.1 Shared Schema (public)

3.1.1 tenants

Column	Type	Constraints	Notes
id	UUID	PK	Auto-generated, used as schema suffix
slug	VARCHAR(50)	UNIQUE NOT NULL	URL-safe identifier e.g. delhi-auto-parts
business_name	VARCHAR(200)	NOT NULL	Display name
contact_name	VARCHAR(100)	NOT NULL	Primary contact person

email	VARCHAR(200)	UNIQUE NOT NULL	Tenant admin login email
phone	VARCHAR(20)		
plan_id	UUID	FK → plans.id	Current subscription plan
status	ENUM	NOT NULL	active | trial | suspended | churned
trial_ends_at	TIMESTAMPTZ	NULLABLE	NULL if not on trial
schema_name	VARCHAR(50)	UNIQUE NOT NULL	e.g. tenant_abc123
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
last_login_at	TIMESTAMPTZ	NULLABLE	Updated on each login
outstanding_method	ENUM	NOT NULL DEFAULT 'direct_upload'	direct_upload: client uploads Busy ledger/outstanding export directly. computed: IIS derives outstanding from sales_invoices + payment_receipts. Configurable per tenant by Tenant Admin in Settings.
inventory_target_weeks	INT	NOT NULL DEFAULT 12	Number of weeks of stock the client wants to hold at all times. Used to compute target stock quantity and suggested order quantity. Configurable in Settings. Default 12 weeks (covers the 3-month procurement lead time).
woi_red_threshold	INT	NOT NULL DEFAULT 4	WOI below this value = Red (critical). Weeks. Configurable in Settings.
woi_amber_threshold	INT	NOT NULL DEFAULT 8	WOI below this value (but >= woi_red_threshold) = Amber (monitor). WOI >= this value = Green (healthy). Configurable in Settings.

> *DEV NOTE: inventory_target_weeks, woi_red_threshold, and woi_amber_threshold are all tenant-level settings configurable in Settings → Inventory Targets. All formulas referencing these values (target stock, suggested order qty, WOI colour thresholds, PO exclusion threshold) must read from tenant settings — never hardcode 12, 4, or 8.*

3.1.2 plans

Column	Type	Constraints	Notes
id	UUID	PK	
name	VARCHAR(50)	NOT NULL	e.g. Starter, Growth, Pro
price_monthly	NUMERIC(10,2)		INR
price_annual	NUMERIC(10,2)		INR

max_users	INT	NOT NULL	
max_skus	INT	NOT NULL	
retention_months	INT	NOT NULL	Data retention period
feature_profitability	BOOLEAN	DEFAULT TRUE	
feature_whatsapp	BOOLEAN	DEFAULT TRUE	
is_active	BOOLEAN	DEFAULT TRUE	

3.2 Tenant Schema Tables

3.2.1 skus (SKU Master)

Column	Type	Constraints	Notes
id	UUID	PK	
sku_code	VARCHAR(100)	UNIQUE NOT NULL	Busy item code
sku_name	VARCHAR(500)	NOT NULL	
category	VARCHAR(200)		Auto-populated from Busy
unit	VARCHAR(20)		e.g. PCS, SET, LTR
is_focus_sku	BOOLEAN	DEFAULT FALSE	User-flagged; never overwritten by import
msl	INT		Last imported MSL from Busy
msl_override	INT	NULLABLE	User-set override; takes precedence over msl
season_tags	JSONB	DEFAULT '[]'	Array of {name, start_month, end_month}
purchase_cost_encoded	VARCHAR(50)	NULLABLE	Raw encoded value from Busy
purchase_cost_decoded	NUMERIC(12,2)	NULLABLE	Computed on import using active formula
last_selling_price	NUMERIC(12,2)	NULLABLE	From latest sales record
is_active	BOOLEAN	DEFAULT TRUE	
created_at	TIMESTAMPTZ	NOT NULL	First seen in any import
updated_at	TIMESTAMPTZ	NOT NULL	

3.2.2 sales

Column	Type	Constraints	Notes
id	UUID	PK	
sku_id	UUID	FK → skus.id	
branch_id	UUID	FK → branches.id NOT NULL	Branch where the sale occurred
sale_date	DATE	NOT NULL	
quantity	NUMERIC(12,3)	NOT NULL	
rate	NUMERIC(12,2)		Selling price per unit
total_value	NUMERIC(14,2)		quantity × rate
customer_id	UUID	FK → customers.id NULLABLE	
import_batch_id	UUID	FK → import_batches.id	Traceability

3.2.3 purchases

Column	Type	Constraints	Notes
id	UUID	PK	
sku_id	UUID	FK → skus.id	
branch_id	UUID	FK → branches.id NOT NULL	Branch that received the purchase
purchase_date	DATE	NOT NULL	
quantity	NUMERIC(12,3)	NOT NULL	
rate_encoded	VARCHAR(50)		Raw encoded purchase cost
rate_decoded	NUMERIC(12,2)	NULLABLE	Decoded at import time
total_value	NUMERIC(14,2)		
vendor_id	UUID	FK → vendors.id NULLABLE	
import_batch_id	UUID	FK → import_batches.id	

3.2.4 inventory_snapshots

Column	Type	Constraints	Notes
id	UUID	PK	

sku_id	UUID	FK → skus.id	
branch_id	UUID	FK → branches.id NOT NULL	Branch this snapshot belongs to
snapshot_date	DATE	NOT NULL	Date of the Busy export
quantity_on_hand	NUMERIC(12,3)	NOT NULL	
import_batch_id	UUID	FK → import_batches.id	

3.2.5 customers

Column	Type	Constraints	Notes
id	UUID	PK	
customer_code	VARCHAR(100)	UNIQUE	From Busy
customer_name	VARCHAR(300)	NOT NULL	
phone	VARCHAR(20)	NULLABLE	Used for WhatsApp wa.me link
whatsapp_number	VARCHAR(20)	NULLABLE	Override if different from phone
created_at	TIMESTAMPTZ	NOT NULL	

3.2.6 sales_invoices

One row per line item on a sales invoice. Imported from the Busy invoice-wise sales report. This table drives both revenue analytics AND outstanding computation. Every invoice line is a debit against the customer.

Column	Type	Constraints	Notes
id	UUID	PK	
invoice_no	VARCHAR(100)	NOT NULL	Busy invoice number. Used to link to payment_receipts.
invoice_date	DATE	NOT NULL	Date invoice was raised
customer_id	UUID	FK → customers.id NOT NULL	
branch_id	UUID	FK → branches.id NOT NULL	Sale From branch
sku_id	UUID	FK → skus.id NOT NULL	One row per line item
quantity	NUMERIC(12,3)	NOT NULL	
rate	NUMERIC(14,2)	NOT NULL	Selling rate per unit
line_amount	NUMERIC(14,2)	NOT NULL	quantity × rate

invoice_total	NUMERIC(14,2)	NOT NULL	Total value of the whole invoice (repeated on each line item row)
import_batch_id	UUID	FK → import_batches.id	

> *DEV NOTE: The existing sales table records SKU-level sales for DRR/forecasting. sales_invoices records the same data but structured invoice-wise for outstanding computation. On import, both tables are populated from the same sales invoice file. invoice_total on each line item row allows the system to compute invoice-level outstanding without aggregating line items every time.*

3.2.7 payment_receipts

One row per payment received against a sales invoice. Imported from the Busy payment receipts / receipt voucher export. Each row reduces the outstanding balance on the referenced invoice.

Column	Type	Constraints	Notes
id	UUID	PK	
receipt_no	VARCHAR(100)	NOT NULL	Busy receipt / voucher number
receipt_date	DATE	NOT NULL	Date payment was received
customer_id	UUID	FK → customers.id NOT NULL	
invoice_no	VARCHAR(100)	NOT NULL	Links to sales_invoices.invoice_no. A single receipt may cover multiple invoices — one row per invoice paid.
amount_received	NUMERIC(14,2)	NOT NULL CHECK > 0	Amount applied to this specific invoice
payment_mode	VARCHAR(50)	NULLABLE	e.g. Cash, NEFT, Cheque — from Busy if available
import_batch_id	UUID	FK → import_batches.id	

> *DEV NOTE: Outstanding balance per invoice = invoice_total (from sales_invoices) − SUM(amount_received WHERE invoice_no = X) from payment_receipts. Outstanding balance per customer = SUM of all their invoice outstanding balances. The ageing bucket is determined by invoice_date of each unpaid/partially-paid invoice, not the payment date.*

3.2.8 outstanding_ledger

Used when tenant.outstanding_method = 'direct_upload'. Stores the raw ledger rows from the Busy outstanding / party ledger export. One row per transaction (invoice raised, payment received, credit note). IIS derives outstanding directly from the net of these rows per customer.

Column	Type	Constraints	Notes
id	UUID	PK	

customer_id	UUID	FK → customers.id NOT NULL	
transaction_date	DATE	NOT NULL	
transaction_type	ENUM	NOT NULL	invoice | payment | credit_note | debit_note
amount	NUMERIC(14,2)	NOT NULL	Positive = debit (invoice/debit note). Negative = credit (payment/credit note).
reference_no	VARCHAR(100)	NULLABLE	Busy invoice or voucher number
narration	TEXT	NULLABLE	Description from Busy ledger if available
import_batch_id	UUID	FK → import_batches.id	

> *DEV NOTE: When outstanding_method = 'direct_upload', the outstanding report reads from outstanding_ledger only. When outstanding_method = 'computed', it reads from sales_invoices + payment_receipts only. The two methods are mutually exclusive per tenant — never mix data sources for a single tenant's outstanding report.*

3.2.7 import_batches

Column	Type	Constraints	Notes
id	UUID	PK	
branch_id	UUID	FK → branches.id NOT NULL	Branch this import belongs to
data_type	ENUM	NOT NULL	sales_invoices | purchases | opening_inventory | payment_receipts | outstanding_ledger | msl | urgent_skus
file_name	VARCHAR(300)	NOT NULL	
file_path	VARCHAR(500)	NOT NULL	S3/R2 path
status	ENUM	NOT NULL	pending | processing | completed | failed
records_total	INT	NULLABLE	
records_imported	INT	NULLABLE	
records_skipped	INT	NULLABLE	
new_masters_created	INT	NULLABLE	SKUs/customers/vendors created
error_log	TEXT	NULLABLE	JSON array of row-level errors
uploaded_by	UUID	FK → tenant_users.id	

created_at	TIMESTAMPTZ	NOT NULL	
completed_at	TIMESTAMPTZ	NULLABLE	

3.2.8 cost_decode_formulas

Column	Type	Constraints	Notes
id	UUID	PK	
char_map	JSONB	NOT NULL	e.g. {"A":"1","B":"2",...}
math_operation	ENUM	NOT NULL	none | divide | multiply | add | subtract
math_value	NUMERIC(10,4)	NULLABLE	e.g. 10 for divide-by-10
is_active	BOOLEAN	DEFAULT TRUE	Only one active formula per tenant
created_by	UUID	FK → tenant_users.id	Tenant Admin only
created_at	TIMESTAMPTZ	NOT NULL	

3.2.9 whatsapp_templates

Column	Type	Constraints	Notes
id	UUID	PK	
template_name	VARCHAR(100)	NOT NULL	
message_body	TEXT	NOT NULL	Supports variables: {customer_name} {outstanding_amount} {due_date} {order_status}
is_default	BOOLEAN	DEFAULT FALSE	Only one default per tenant
created_at	TIMESTAMPTZ	NOT NULL	

3.2.10 forecasting_cache

Computed nightly / on import. Stored per branch AND at consolidated level (branch_id = NULL means consolidated/all-branches).

Column	Type	Constraints	Notes
id	UUID	PK	
sku_id	UUID	FK → skus.id	
branch_id	UUID	FK → branches.id NULLABLE	NULL = consolidated across all branches

computed_at	TIMESTAMPTZ	NOT NULL	
drr_4w	NUMERIC(10,4)		Daily run rate, last 4 weeks
drr_13w	NUMERIC(10,4)		Daily run rate, last 13 weeks
drr_52w	NUMERIC(10,4)		Daily run rate, last 52 weeks
drr_recommended	NUMERIC(10,4)		Weighted blend; used for order calcs
drr_seasonal	NUMERIC(10,4)	NULLABLE	Only set if seasonal window is active/upcoming
seasonal_uplift_pct	NUMERIC(8,2)	NULLABLE	
woi	NUMERIC(8,2)		Weeks of inventory
woi_status	ENUM	NOT NULL	red | amber | green
msl_suggested	INT		System-recommended MSL
target_stock_qty	INT		Stock target quantity = CEIL(DRR_recommended × inventory_target_weeks × 7). Recomputed whenever inventory_target_weeks setting changes.
suggested_order_qty	INT		
pre_season_alert	BOOLEAN	DEFAULT FALSE	True if season starts within 20 weeks
latest_order_date	DATE	NULLABLE	Season start minus lead time

3.2.11 branches

One record per physical branch/location. Branches can be created manually by the Tenant Admin or auto-created during import when a 'Sale From' / location column is detected in the Busy export file. Every tenant has at least one branch (Home Branch) created automatically at provisioning.

Column	Type	Constraints	Notes
id	UUID	PK	
branch_code	VARCHAR(20)	UNIQUE NOT NULL	Short identifier e.g. HQ, STORE, WH. URL-safe. Auto-generated from source_label if auto-created.
branch_name	VARCHAR(200)	NOT NULL	Display name. Editable. If auto-created, initialised from source_label value (e.g. 'WAREHOUSE' → 'Warehouse').
address	TEXT	NULLABLE	
is_home_branch	BOOLEAN	DEFAULT FALSE	Only one per tenant. Cannot be deleted.
is_active	BOOLEAN	DEFAULT TRUE	Inactive branches hidden from selectors but data preserved

source_label	VARCHAR(200)	NULLABLE	Raw value from Busy export that this branch was auto-created from (e.g. 'WAREHOUSE', 'STORE', 'Shop 1'). NULL for manually created branches. Used to match rows on subsequent imports without re-mapping.
auto_created	BOOLEAN	DEFAULT FALSE	TRUE if branch was created automatically from import data. FALSE if created manually by admin.
created_at	TIMESTAMPTZ	NOT NULL	

3.2.12 branch_column_maps

Remembers which column in a Busy export file contains the Sale From / location identifier, per tenant and per file format. Prevents the user from having to re-configure on every upload.

Column	Type	Constraints	Notes
id	UUID	PK	
data_type	ENUM	NOT NULL	sales | purchases | inventory — the import type this mapping applies to
column_name	VARCHAR(200)	NOT NULL	Exact column header from the Busy file e.g. 'Sale From', 'Location', 'Branch'
confirmed_by	UUID	FK → tenant_users.id	User who confirmed the mapping on first detection
created_at	TIMESTAMPTZ	NOT NULL	
updated_at	TIMESTAMPTZ	NOT NULL	

> *DEV NOTE: One branch_column_maps record per data_type per tenant. If a tenant uploads a sales file and confirms 'Sale From' as the location column, that mapping is saved. Future sales uploads use it automatically — no prompt needed unless the column is absent.*

3.2.12 stock_transfers

Records movement of a SKU from one branch to another. Both source and destination must be branches of the same tenant.

Column	Type	Constraints	Notes
id	UUID	PK	
transfer_date	DATE	NOT NULL	
sku_id	UUID	FK → skus.id NOT NULL	
from_branch_id	UUID	FK → branches.id NOT NULL	Source branch

to_branch_id	UUID	FK → branches.id NOT NULL	Destination branch; must differ from from_branch_id
quantity	NUMERIC(12,3)	NOT NULL CHECK > 0	
notes	TEXT	NULLABLE	Optional reason or reference
created_by	UUID	FK → tenant_users.id	
created_at	TIMESTAMPTZ	NOT NULL	

> *DEV NOTE: stock_transfers do NOT directly mutate inventory_snapshots. The inventory computation layer must account for transfers when calculating current stock: effective_stock = last_snapshot_qty + purchases_since_snapshot - sales_since_snapshot + transfers_in - transfers_out.*

4. Module: Data Import & Management

4.1 User Stories

US-ID	As a…	I want to…	So that…
US-IMP-01	Tenant User	upload Busy export files (sales invoices, purchases, payment receipts or outstanding ledger, opening inventory, MSL) and have each row processed correctly	the system handles all data formats and computes outstanding via whichever method the tenant has configured
US-IMP-02	Tenant User	see a validation summary after upload showing records imported, skipped, branches detected, and any errors	I can verify data quality and confirm branch assignments before relying on reports
US-IMP-03	Tenant User	upload an Urgent SKU list in Excel	the system generates an order report for those specific SKUs
US-IMP-04	Tenant Admin	see a full history of all past imports with status, record counts, and branch breakdown	I can audit what data is in the system and how it was distributed across branches

US-IMP-05	System (Auto)	auto-create master records (SKUs, customers, vendors) from first import	users never have to manually set up masters
US-IMP-06	System (Auto)	never overwrite user-defined flags (Focus SKU, Season Tags, MSL overrides) on subsequent imports	user configurations are preserved
US-IMP-07	System (Auto)	detect the Sale From / location column in a Busy export and auto-create branches from its distinct values	branches are set up from real data without any manual configuration
US-IMP-08	Tenant Admin	confirm the Sale From column mapping once — and never be asked again for the same file format	repeat uploads are frictionless
US-IMP-09	Tenant Admin	manually override which branch a specific import batch is assigned to, if auto-detection assigns it incorrectly	I can correct mistakes without re-uploading the file
US-IMP-10	Tenant Admin	choose between 'Direct Upload' and 'Computed' as my outstanding method in Settings	I can pick the method that matches how my Busy data is available

Urgent SKU Handling in PO Advisor — Confirmed Decision

There is no separate Urgent SKU screen. Urgent SKUs uploaded by the user appear highlighted within the PO Advisor Last 7 Days tab (same as Smart Reorder Bucket 1). Confirmed in v2.9.

5. Urgent SKUs with a sale in last 7 days: full DRR analysis run; SKU appears with URGENT badge in the Last 7 Days tab.
6. Urgent SKUs with no sale in last 7 days: URGENT badge with note: No recent sale — order quantity estimated from 13W DRR trend.
7. Urgent SKUs with no sale history: URGENT badge with Insufficient history — manual quantity required.
8. Order qty formula: MAX(0, DRR_recommended × lead_time_days × 1.20 − effective_stock).
9. Mark as Order Noted clears the URGENT badge for that SKU.

> *DEV NOTE: On urgent_skus import store in import_batches with data_type=urgent_skus. On PO Advisor Last 7 Days load join most recent urgent_skus batch (created_at >= 7 days ago). Tag matching SKUs urgent=true. No separate /po-advisor/urgent route.*

4.2 Pages

PAGE: Data Import — Upload	
URL / Route	/import/upload
Access Roles	tenant_admin, tenant_user

Description	Upload Busy export files or the Urgent SKU list. Displays upload zones per data type.

Upload Zones (one per data type)

Each zone is a drag-and-drop area with a file-picker fallback. Zones:

- Sales Invoices (label: 'Sales Invoice-wise — invoice no., date, customer, SKU, qty, rate, amount, Sale From')
- Purchase Invoices (label: 'Purchase Invoice-wise — invoice no., date, vendor, SKU, qty, rate, amount')
- Payment Receipts (label: 'Payment receipts — receipt no., date, customer, invoice no., amount received'). Shown only when outstanding_method = 'computed'.
- Customer Outstanding / Ledger (label: 'Busy party ledger / outstanding export — customer, date, type, amount, reference'). Shown only when outstanding_method = 'direct_upload'.
- Opening Inventory (label: 'One-time opening stock snapshot — SKU-wise closing stock'. Shown with a prominent ONE-TIME badge.)
- MSL / Reorder Levels (label: 'Current MSL settings from Busy — SKU, reorder level')
- Urgent SKU List (label: 'Urgent SKU Excel — SKU Code, Name, [Priority]')

> *DEV NOTE: The 'Payment Receipts' zone and the 'Customer Outstanding / Ledger' zone are mutually exclusive — only one is shown at a time based on the tenant's outstanding_method setting (configurable in Settings → Outstanding Method). Default is 'direct_upload'. The setting can be changed at any time; switching method does not delete existing data but the outstanding report will switch to reading from the new source immediately.*

Field Name	Type	Required	Validation / Notes	Source
file	File	Yes	Accepts .csv, .xlsx only. Max size: 10 MB.	User upload
branch_id	Select	Conditional	Shown only when no Sale From column is detected in the file AND no saved branch_column_map exists for this data type. Dropdown of active branches; pre-selects Home Branch. When auto-detection is active, this field is hidden — branches are assigned per-row from the file.	User selection / auto
data_type	Enum (auto)	Yes	Auto-detected from zone; user can override via dropdown	UI zone / user selection
import_notes	Text	No	Optional note e.g. 'Feb 2025 data'	User input

Branch Detection Flow on Upload

10. When the file is submitted, the background parser first scans column headers before processing any data rows (see Section 4.3.3 for full logic).
11. If a Sale From column is found and a saved mapping exists: import proceeds automatically — rows are tagged to branches with no user interaction needed.
12. If a Sale From column is found but NO saved mapping exists: import is paused and a one-time confirmation banner is shown on the Import History page: 'We found a [Sale From] column in your file — should we use it to identify branches?' User confirms once; mapping is saved for all future uploads of the same data type.
13. If no Sale From column is found: the branch_id the user selected at upload time is applied to all rows.

Behaviour on Submit

14. File is uploaded to S3 and an import_batch record created with status='pending'.
15. A background job picks it up within 5 seconds.
16. User is redirected to the Import History page with a live status indicator.
17. On completion, a toast notification appears: 'Import complete: X records imported, Y skipped.'

> *DEV NOTE: File parsing should happen in a background worker, NOT in the request-response cycle. Large CSV files can have 10,000+ rows.*

PAGE: Data Import — History	
URL / Route	/import/history
Access Roles	tenant_admin, tenant_user
Description	Full history of all import batches for this tenant, newest first.

Table Columns

Column	Type	Sortable	Filterable	Notes
File Name	Text	Yes	No	Clickable to download original file
Branch	Badge	No	Yes	Which branch this import belongs to
Data Type	Badge	No	Yes	e.g. Sales, Purchase, Inventory
Status	Badge	No	Yes	pending|processing|completed|failed
Records Total	Number	Yes	No	
Records Imported	Number	Yes	No	
Records Skipped	Number	Yes	No	Clickable to view error log
New Masters Created	Number	No	No	SKUs/customers/vendors/branches auto-created
Uploaded By	Text	No	No	

Date/Time	DateTime	Yes	No	User's timezone	

Error Log Modal

Clicking 'Records Skipped' opens a modal showing a table of skipped rows: Row Number | Raw Data | Error Reason.

4.3 Import Processing Logic

4.3.1 Column Mapping

The system must handle common Busy export column name variations:

Data Type	Expected Columns	Fallback / Aliases
Sales Invoices	Invoice No, Invoice Date, Party Name, Item Code, Item Name, Qty, Rate, Amount	Bill No / Voucher No, Date, Customer / Party, SKU Code, Quantity, Price, Value
Sale From / Location	Sale From	Location, Branch, Godown, Store, Warehouse, Sale Point, Dispatched From
Purchase Invoices	Invoice No, Invoice Date, Party Name, Item Code, Item Name, Qty, Rate, Amount	Bill No, Date, Vendor / Supplier, SKU Code, Quantity, Price, Value
Purchase Location	Purchase At / Received At	Location, Branch, Godown
Payment Receipts	Receipt No, Receipt Date, Party Name, Invoice No, Amount	Voucher No, Date, Customer / Party, Bill No / Against Invoice, Amount Received
Outstanding / Ledger	Party Name, Date, Debit, Credit OR Amount + Type	Customer, Transaction Date, Dr, Cr, Invoice No / Voucher No, Narration
Opening Inventory	Item Code, Item Name, Closing Stock	SKU Code, Stock, Balance Qty, Opening Stock
MSL	Item Code, Reorder Level	Min Stock, MSL
Urgent SKUs	Item Code, Item Name	SKU Code, Product Name

4.3.2 Master Auto-Creation Rules

18. Sales Invoices import: each row populates both the sales table (for DRR/forecasting) AND the sales_invoices table (for outstanding computation when method = 'computed'). On first import, unrecognised Item Codes auto-create skus records; unrecognised Party Names auto-create customers records.
19. Purchase Invoices import: each row populates the purchases table. On first import, unrecognised Party Names auto-create vendors records.
20. Payment Receipts import (method = 'computed' only): rows matched to existing customers by Party Name. Auto-create customer if not found. Receipt rows matched to sales_invoices by invoice_no — unmatched invoice numbers logged as warnings, not errors.

21. Outstanding / Ledger import (method = 'direct_upload' only): rows stored in outstanding_ledger. Party Names auto-create customers if not found. Rows are upserted by reference_no + transaction_date + customer_id to prevent duplicates on re-upload.
22. Opening Inventory import: one-time. Creates inventory_snapshots records. Does NOT auto-create SKUs — SKUs must already exist from a prior sales import. Unrecognised Item Codes flagged as warnings.
23. On subsequent imports: never overwrite is_focus_sku, season_tags, msl_override on existing SKU records.
24. After each sales or purchase import, trigger background forecasting recomputation for all affected SKUs.

4.3.3 Sale From / Location Auto-Detection & Branch Auto-Creation

This is the primary mechanism by which branches are identified from real Busy export data. The system detects location information directly from invoice rows without requiring the user to pre-configure branches manually.

Step 1 — Column Detection on Upload

During file parsing (background job), before processing any data rows, the parser scans the file headers for known Sale From column names (see column mapping table in 4.3.1). Detection priority order:

25. Check branch_column_maps table: if a confirmed mapping already exists for this data_type, use that column name directly. No prompt shown to user.
26. If no saved mapping: scan headers for known aliases (Sale From, Location, Branch, Godown, Store, Warehouse, Sale Point, Dispatched From). Case-insensitive match.
27. If a candidate column is found: pause import and surface a one-time confirmation prompt to the user: 'We found a column [Sale From] in your file. Should we use this to identify branches? [Yes, use this column] [No, assign all rows to one branch]'. Store the confirmed column name in branch_column_maps.
28. If no location column is found and no mapping exists: proceed with the branch_id the user selected at upload time (single-branch import).

Step 2 — Branch Auto-Creation from Distinct Values

Once the location column is confirmed (either from saved mapping or user confirmation):

29. Read all distinct values from that column in the file (e.g. 'STORE', 'WAREHOUSE', 'Main Shop').
30. For each distinct value: check branches table for a row where source_label = that value (case-insensitive). If found: use that branch_id. If NOT found: auto-create a new branch record with: branch_name = title-cased value (e.g. 'WAREHOUSE' → 'Warehouse'), source_label = raw value, auto_created = TRUE, branch_code = auto-generated slug (e.g. 'WAREHOUSE' → 'WH', 'STORE' → 'STORE'). Log new branch creation in import summary.
31. Every data row is then tagged with the branch_id that matches its Sale From value.

> *DEV NOTE: Pilot tenant example — Busy export contains column 'Sale From' with values 'Store' and 'Warehouse'. On first upload: system detects column, asks user to confirm, creates two branches: branch_name='Store' (source_label='Store') and branch_name='Warehouse' (source_label='Warehouse'). On all future uploads: system reads branch_column_maps, finds 'Sale From', looks up source_label matches, and tags rows automatically — zero user interaction.*

Step 3 — Conflict Resolution

32. If a row has a Sale From value that is blank or NULL: assign to the tenant's Home Branch. Log as a warning in the import error log.
33. If a previously seen source_label now maps to a branch that has been deactivated: re-activate the branch automatically and notify the admin via a toast: 'Branch [Warehouse] was reactivated because new data was found for it.'
34. If the user has manually renamed a branch (e.g. 'Warehouse' renamed to 'Main Godown'): the source_label matching still works because source_label is preserved as the raw value from Busy. The display name is independent.

Step 4 — Import Summary — Branch Creation Report

The import completion notification and the Import History detail view must show:

- Branches detected in this file: [list of branch names]
- New branches auto-created: [count + names] (shown only if > 0)
- Rows assigned per branch: e.g. Store: 842 rows | Warehouse: 317 rows

Manual Branch Creation — Always Available

The above auto-detection flow is the default and recommended path. Manual branch creation (via Settings → Branches) remains fully available for cases where:

- The client wants to set up branches before uploading any data.
- A branch exists physically but has no Busy data yet (e.g. a new location just opened).
- The Busy export does not contain a Sale From column and branches need to be created ahead of time.

Manually created branches have auto_created = FALSE and source_label = NULL. On a subsequent import, if a Sale From value in the file matches the branch_name of a manually created branch (case-insensitive), the system will link them and populate source_label automatically.

4b. Module: Branch & Location Management

IIS supports businesses with multiple physical locations — shops, warehouses, satellite stores — all under a single tenant account. Branch data is always fully scoped to the tenant; no cross-tenant visibility is ever permitted.

4b.1 User Stories

US-ID	As a…	I want to…	So that…
US-BRN-01	System (Auto)	detect distinct Sale From values in a Busy export and auto-create branches from them	branches are created from real invoice data with no manual setup required
US-BRN-02	Tenant Admin	create a branch manually before any data is uploaded for it	I can set up a new location in

			advance of its first import
US-BRN-03	Tenant Admin	rename an auto-created branch to a friendlier display name without breaking Sale From matching	I can change 'WAREHOUSE' to 'Main Godown' and future imports still route correctly
US-BRN-04	Tenant Admin	set one branch as the Home Branch	blank or unrecognised Sale From rows are assigned here and it is pre-selected on all screens
US-BRN-05	Tenant User	view stock levels, WOI, and DRR for a specific branch	I can monitor each location independently
US-BRN-06	Tenant User	switch to a Consolidated view that aggregates all branches	I can see the total business picture at once
US-BRN-07	Tenant Admin	record a stock transfer from one branch to another	I can account for inter-branch stock movements accurately
US-BRN-08	Tenant User	view the full stock transfer log filterable by branch, SKU, and date	I have a complete audit trail of all inter-branch movements
US-BRN-09	Tenant User	filter any report (inventory, PO, profitability, seasonal) by branch	I can generate branch-specific reports for the relevant manager
US-BRN-10	Tenant User	see stock, DRR, and WOI for all branches side by side on one screen	I can spot which branch is understocked and decide whether to transfer or order
US-BRN-11	Tenant User	see sales volume and revenue for each branch side by side for a selected period	I can compare how much each location is selling
US-BRN-12	Tenant Admin	see gross margin % and gross profit for each branch side by side	I can identify whether one branch is more

			profitable than another
US-BRN-13	Tenant User	see the top-selling SKUs for each branch ranked side by side	I can see whether branches have different best-sellers and make branch-specific buying decisions
US-BRN-14	Tenant User	instantly see a Transfer Opportunity flag when one branch is out-of-stock while another has surplus	I can fix imbalances internally before placing a vendor order

4b.2 Pages

PAGE: Branch Management	
URL / Route	/settings/branches
Access Roles	tenant_admin
Description	View, create, rename, and deactivate branches. Both manually created and auto-created branches appear here. Auto-created branches are badged 'Auto' with their source label shown so the admin can see exactly which Sale From value they came from.

Branch List Table

Branch Name	Code	Source Label	Created By	Status	Home	Actions

Source Label column: shows the raw Busy 'Sale From' value (e.g. 'WAREHOUSE') for auto-created branches. Shows '—' for manually created branches. Hovering shows a tooltip: 'Auto-created from Sale From column on [date]'.

'Created By' column: shows 'Auto (Import)' for auto-created branches; shows the admin's name for manually created ones.

Create / Edit Branch — Fields

Field Name	Type	Required	Validation / Notes	Source
branch_name	Text	Yes	Max 200 chars. Must be unique within tenant.	User input
branch_code	Text	Yes	Max 20 chars. Alphanumeric + hyphens only. Auto-uppercased. Must be unique within tenant.	User input

address	Textarea	No	Free text address for reference.	User input
is_active	Toggle	Yes	Inactive branches are hidden from all dropdowns but their historical data is preserved.	User toggle

Rules

- Home Branch cannot be deactivated or deleted. To change the Home Branch, admin selects another branch and marks it as Home — the old Home Branch is demoted automatically.
- A branch with any imported data cannot be deleted — only deactivated.
- Renaming an auto-created branch (e.g. 'WAREHOUSE' → 'Main Godown') updates branch_name only. source_label is never changed — it remains 'WAREHOUSE' so that future imports still match correctly without any reconfiguration.
- If the admin wants to merge two auto-created branches that are actually the same location (e.g. 'WAREHOUSE' and 'Warehouse' created as duplicates due to capitalisation differences in Busy), they can select one as the primary and reassign the other's data to it. The secondary branch is then deactivated.

> *DEV NOTE: source_label matching during import is always case-insensitive and whitespace-trimmed. This prevents duplicate branch creation from trivial formatting differences in Busy exports (e.g. 'Store' vs 'STORE' vs ' Store').*

PAGE: Stock Transfer — New	
URL / Route	/branches/transfer/new
Access Roles	tenant_admin
Description	Record a manual inter-branch stock transfer for one SKU.

Field Name	Type	Required	Validation / Notes	Source
transfer_date	Date	Yes	Defaults to today. Cannot be future-dated.	User input
sku_id	Select + Search	Yes	Search by SKU code or name.	User selection
from_branch_id	Select	Yes	Dropdown of active branches. Cannot equal to_branch_id.	User selection
to_branch_id	Select	Yes	Dropdown of active branches. Cannot equal from_branch_id.	User selection
quantity	Number	Yes	Must be > 0. Must be <= current stock at from_branch. Decimal allowed.	User input

notes	Text	No	Max 500 chars. Optional reference or reason.	User input

Validation

35. Warn (not block) if quantity > current stock at from_branch. Show current stock figure alongside the field.
36. from_branch_id and to_branch_id must be different branches of the same tenant.
37. On save: write to stock_transfers table. Trigger forecasting recomputation for affected SKU at both branches.

PAGE: Stock Transfer Log	
URL / Route	/branches/transfer/log
Access Roles	tenant_admin, tenant_user
Description	View all inter-branch stock transfers. Filterable, sortable, exportable.

Filters

Field Name	Type	Required	Validation / Notes	Source
filter_branch	Select	No	All | [Branch Name] — filters on either from_branch or to_branch	User selection
filter_sku	Text	No	Search by SKU code or name	User input
date_from	Date	No		User input
date_to	Date	No		User input

Table Columns

Date	SKU Name	From Branch	To Branch	Qty	By	Notes

PAGE: Cross-Branch Comparison	
URL / Route	/branches/comparison
Access Roles	tenant_admin, tenant_user
Description	Comprehensive side-by-side analytics across all active branches covering inventory health, sales performance, and profitability. All four views are tabs on the same page — branch columns are dynamic and driven by the active branches list.

> *DEV NOTE: Number of branch columns is always dynamic — generated at query time from the active branches list. Use a pivot/transpose query or application-level transpose for all four tabs. Never hard-code branch columns.*

Global Controls (apply to all tabs)

Field Name	Type	Required	Validation / Notes	Source
filter_sku	Text / Multi-select	No	Search by SKU code or name. Leave blank to show all SKUs. Applies across all tabs.	User input
date_period	Select	No	Last 4 Weeks | Last 13 Weeks (default) | Last 26 Weeks | Last 52 Weeks. Applies to sales and profitability tabs.	User selection
filter_woi_status	Select	No	All | Red | Amber | Green. Applies to Inventory tab only.	User selection

Tab 1 — Inventory Comparison (Stock / DRR / WOI)

Columns: SKU Code | SKU Name | [Per Branch: Stock | DRR | WOI (colour-coded)] | Consolidated Total Stock

- Each branch gets three sub-columns: Stock, DRR (Recommended), WOI.
- WOI cell background: red if < 4w, amber if 4–8w, green if >= 8w — independently per branch.
- Consolidated Total Stock = sum of effective_stock across all active branches.
- Sort options: SKU Name | Worst WOI (any branch) | Best Consolidated Stock.
- 'Transfer Opportunity' flag column: shows a transfer icon on rows where at least one branch is Red WOI AND at least one other branch is Green WOI for the same SKU. Clicking opens the Stock Transfer form pre-filled with that SKU.

Tab 2 — Sales Comparison (Volume & Revenue)

Columns: SKU Code | SKU Name | [Per Branch: Units Sold | Revenue (₹)] | Total Units | Total Revenue | Branch with Highest Sales

- Period controlled by the date_period selector above.
- Revenue = SUM(sales.total_value) per branch per SKU for the selected period.
- Units Sold = SUM(sales.quantity) per branch per SKU for the selected period.
- 'Branch with Highest Sales' column highlights which branch sold the most units of each SKU — useful to spot demand concentration.
- Summary row at bottom: total units and revenue per branch across all shown SKUs.
- A branch sales share bar is shown below each revenue cell: e.g. Branch A = 65% | Branch B = 35% of total revenue for that SKU, shown as a mini horizontal bar.
- Sort options: SKU Name | Total Revenue DESC | Total Units DESC | Most Uneven Distribution (branch with highest variance across branches).

> *DEV NOTE: Data source is sales table filtered by branch_id and date range. Revenue is total_value; units is quantity. Aggregate per (sku_id, branch_id) then pivot to columns.*

Tab 3 — Profitability Comparison (Margin % & Gross Profit)

Columns: SKU Code | SKU Name | [Per Branch: Gross Margin % | Gross Profit (₹) | Revenue (₹)] | Best Margin Branch

- Gross Margin % per branch = (SUM(revenue) - SUM(decoded_cost × qty)) / SUM(revenue) × 100 for the selected period.
- Gross Profit (₹) per branch = SUM(revenue) - SUM(decoded_cost × qty) for the selected period.
- 'Best Margin Branch' column shows which branch achieved the highest gross margin % for each SKU.
- Cells where margin < 10% are highlighted amber; cells where margin < 0% are highlighted red (selling below cost).
- Volume-Profit Divergence flag: shown on rows where a branch has top-20% sales volume but below-median margin — same logic as the profitability module but computed per branch.
- Accessible to tenant_admin only. Gross margin column hidden for tenant_user.
- Sort options: SKU Name | Best Gross Profit (consolidated) | Worst Margin (any branch).

> *DEV NOTE: Decoded cost (rate_decoded) comes from purchases table — use most recent purchase record per SKU as the cost basis, same as the profitability module. This tab must enforce the same access restriction: tenant_user role receives 403 for the /branches/comparison?tab=profitability endpoint.*

Tab 4 — Top SKUs by Branch

A ranked view: for each branch, show the top N SKUs by sales volume and by revenue for the selected period. Allows direct side-by-side comparison of what each branch's best sellers are.

- Layout: one panel per branch (horizontally scrollable if many branches). Each panel shows: Rank | SKU Code | SKU Name | Units Sold | Revenue | Margin %.
- Top N selector: 10 / 20 / 50 SKUs (default 10).
- SKUs that appear in the top N for multiple branches are highlighted with a 'Common Top SKU' badge.
- SKUs that appear in Branch A's top N but NOT in Branch B's top N are flagged — this surfaces demand patterns that differ between locations.
- Export: exports all branch panels into a single Excel file with one sheet per branch.

> *DEV NOTE: Query: for each branch, SELECT sku_id, SUM(quantity) as units, SUM(total_value) as revenue FROM sales WHERE branch_id = X AND sale_date >= period_start GROUP BY sku_id ORDER BY units DESC LIMIT N. Join skus for name and profitability for margin. Run once per branch, merge results in application layer.*

Export

- Tab 1 (Inventory): Excel — one row per SKU, one column group per branch (Stock, DRR, WOI), plus Consolidated.
- Tab 2 (Sales): Excel — one row per SKU, one column group per branch (Units, Revenue), plus Total and Share %.
- Tab 3 (Profitability): Excel — one row per SKU, one column group per branch (Margin %, Gross Profit, Revenue). Admin only.
- Tab 4 (Top SKUs): Excel — one sheet per branch, ranked list.
- All exports include the date period and branch list in the header row.

4b.3 Branch Context in All Modules

Every module that shows inventory or sales data must respect the active branch context selected by the user. The branch context is held in a session-level state variable (not a URL parameter, to avoid copy-paste confusion). The global header always shows the active branch name.

Module	Branch-Specific Behaviour	Consolidated Behaviour
Inventory Health	Shows stock/WOI/DRR for selected branch only	Sums stock across all branches; DRR aggregated by total sales
PO Advisor	Order qty computed to bring selected branch to 12-week target	Order qty computed on consolidated stock — single order recommendation
Seasonal Planning	Pre-season alert uses selected branch's stock for gap calc	Consolidated stock used for gap calc
Profitability	Sales and cost data filtered to selected branch	All-branch aggregation
Customer Outstanding	Outstanding is tenant-level — not branch-specific	Same as branch-specific (no difference)
Dashboard	All widgets scoped to selected branch	All widgets show consolidated figures
Data Import	User selects branch at upload time	N/A — import always tags a branch

4b.4 Effective Stock Calculation with Transfers

Because Busy exports give a stock snapshot (point-in-time), and transfers happen inside IIS (not in Busy), the effective current stock must be computed as:

```
effective_stock(sku, branch, as_of_date) =
  latest_inventory_snapshot(sku, branch, <= as_of_date).quantity_on_hand
  + SUM(purchases.quantity WHERE branch = branch AND date > snapshot_date AND date <= as_of_date)
  - SUM(sales.quantity    WHERE branch = branch AND date > snapshot_date AND date <= as_of_date)
  + SUM(stock_transfers.quantity WHERE to_branch   = branch AND date > snapshot_date AND date <= as_of_date)
  - SUM(stock_transfers.quantity WHERE from_branch = branch AND date > snapshot_date AND date <= as_of_date)
```

> *DEV NOTE: This formula is the authoritative definition of current stock. Wherever 'current stock' is displayed or used in calculations (WOI, order qty, pre-season gap), this formula must be used — not just the raw snapshot. Create a database view or a reusable service function for this.*

4A. Busy Web Service Integration

IIS integrates with Busy Accounting Software via Busy's built-in Web Service. When the Web Service is enabled in Busy, Busy acts as a local HTTP server listening on port 981. IIS sends HTTP GET requests to this endpoint

with specific Service Codes (SC) as request headers. Busy executes the request and returns results as XML in the response body.

This integration completely eliminates manual CSV/Excel exports. Data flows automatically from Busy into IIS on a scheduled basis — no user action required after initial setup.

> *DEV NOTE: Busy Cloud and IIS will both be hosted on the same private cloud VPC. This means the Busy Web Service endpoint is reachable from the IIS Python API via a private internal IP (e.g., http://192.168.x.x:981). Port 981 must be open within the private subnet only — never exposed to the public internet.*

4A.1 Busy Service Codes Reference

Busy's Web Service supports 9 Service Codes. IIS uses only Service Code 1 (SQL Query) for all data retrieval. All other codes are write operations not needed by IIS.

Service Code 1 — GetXML from Recordset (READ — used by IIS)

Executes any SQL query against the open Busy company database. Returns results as an XML string in the response body. This is the only service code IIS uses.

Request headers: SC=1 | Qry={SQL query string} | UserName={Busy company username} | Pwd={Busy company password}

Response header Result='T' = success, Result='F' = failure (error in Description header). Response body = XML recordset.

Other Service Codes (write operations — NOT used by IIS)

- SC=2: Add Voucher from XML
- SC=3: Modify Voucher by Voucher No
- SC=4: Modify Voucher by Voucher Code
- SC=5: Add Master from XML
- SC=6: Modify Master by Code
- SC=7: Modify Master by Name
- SC=8: Get Voucher XML by VchCode
- SC=9: Get Master XML by MasterCode

> *DEV NOTE: IIS is a read-only consumer of Busy data. IIS never writes back to Busy. SC=1 covers all IIS data retrieval needs. Do not implement SC=2 through SC=9 in the IIS integration layer.*

4A.2 Busy VchType and MasterType Constants

These are the official integer constants from Busy's documentation. Use these exact values in all SQL queries against Busy's Tran1 table (VchType column) and Mast1/Item1 tables.

Voucher Types (VchType) — used in Tran1 table

- PURCHASE = 2
- SALE_RETURN = 3
- STOCK_TRANSFER = 5
- SALE = 9
- PURCHASE_RETURN = 10
- SALE_ORDER = 12
- PURCHASE_ORDER = 13
- RECEIPT = 14 (payment received from customer)
- PAYMENT = 19 (payment made to vendor)
- DEBIT_NOTE = 17
- CREDIT_NOTE = 18

- PHYSICAL_STOCK = 61

Master Types (MasterType) — used for master data fetch

- ACCOUNT_GROUP_MASTER = 1
- ACCOUNT_MASTER = 2 (customers and vendors)
- ITEM_GROUP_MASTER = 5
- ITEM_MASTER = 6 (SKU / product catalogue)
- UNIT_MASTER = 8
- MATERIAL_CENTER_MASTER = 11 (godowns / warehouses)
- BRANCH_MASTER = 24 (branches / locations)
- CONTACT_MASTER = 36

> *DEV NOTE: These constants are from Busy's official documentation (Master and Voucher types.pdf). Never hardcode plain integers in query logic — always use named constants (e.g., BUSY_VCHTYPE_SALE = 9). This makes queries self-documenting and resistant to accidental errors.*

4A.3 Data Sync Design — Full Sync and Delta Sync

IIS uses a two-phase sync strategy. Phase 1 (Full Sync) runs once when a tenant first connects Busy. Phase 2 (Delta Sync) runs on a nightly schedule thereafter, fetching only records created or modified since the last successful sync.

4A.3.1 Phase 1 — Initial Full Sync

Triggered manually by the Tenant Admin from Settings → Busy Integration → 'Run Initial Sync'. Fetches the entire history from Busy with no date filter. This may take several minutes for 2+ years of data. A progress indicator is shown. The sync runs as a background job.

Data fetched during Full Sync:

- Item Master (ITEM_MASTER = 6) — all SKUs / products
- Account Master — Sundry Debtors group (ACCOUNT_MASTER = 2) — all customers
- Account Master — Sundry Creditors group (ACCOUNT_MASTER = 2) — all vendors
- Material Center Master (MATERIAL_CENTER_MASTER = 11) — all godowns / warehouses
- Branch Master (BRANCH_MASTER = 24) — all branches
- Sales vouchers (VchType = 9) — all time
- Purchase vouchers (VchType = 2) — all time
- Sale Return vouchers (VchType = 3) — all time
- Purchase Return vouchers (VchType = 10) — all time
- Receipt vouchers (VchType = 14) — all time — for outstanding Method B (computed)
- Physical Stock vouchers (VchType = 61) — for opening stock snapshot
- Stock Transfer vouchers (VchType = 5) — all time

4A.3.2 Phase 2 — Incremental Delta Sync

Runs automatically every night at 23:00 (configurable per tenant). Fetches only records created or modified after the last successful sync timestamp. The sync window is: from_dt = last successful sync completed_at, to_dt = NOW() at time of sync run.

Delta sync schedule by data type:

- Transactions (Sales, Purchases, Returns, Receipts, Stock Transfers): nightly at 23:00
- Masters (Items, Customers, Vendors, Godowns): weekly on Sunday at 01:00 — masters change rarely; weekly is sufficient
- Physical Stock snapshot: nightly at 23:00 — captures closing stock position for the day

> *DEV NOTE: Masters do not have a reliable 'modified date' column in all Busy versions. To handle this safely, master sync fetches ALL master records weekly and uses upsert (INSERT ... ON CONFLICT DO UPDATE). Since master counts are small (hundreds to low thousands), a full weekly master sync is fast and avoids missed updates entirely.*

4A.3.3 sync_log Table

Every sync run (full or delta, success or failure) is recorded in the sync_log table. This is the authoritative source for 'what data does IIS currently have from Busy' and drives the from_dt calculation for the next delta sync.

```
CREATE TABLE {tenant_schema}.sync_log (
    id                SERIAL PRIMARY KEY,
    sync_type         TEXT NOT NULL,         -- 'full' | 'delta'
    data_type         TEXT NOT NULL,         -- 'sales' | 'purchases' | 'items' |
etc.
    from_dt           TIMESTAMPTZ,           -- NULL for full sync
    to_dt             TIMESTAMPTZ NOT NULL,
    records_fetched   INT DEFAULT 0,
    records_inserted  INT DEFAULT 0,
    records_updated   INT DEFAULT 0,
    records_skipped   INT DEFAULT 0,
    status            TEXT NOT NULL DEFAULT 'running',  -- 'running' | 'success' |
'failed'
    error_message     TEXT,
    started_at        TIMESTAMPTZ DEFAULT NOW(),
    completed_at      TIMESTAMPTZ
);
```

> *DEV NOTE: The next delta sync's from_dt = SELECT MAX(completed_at) FROM sync_log WHERE data_type = '{type}' AND status = 'success'. If no successful sync exists for a data_type, treat it as a full sync (no date filter). Never rely on a failed sync's to_dt — only successful syncs advance the watermark.*

4A.3.4 Upsert Strategy — No Duplicate Data

All inserts into IIS tables from Busy data use upsert (INSERT ... ON CONFLICT DO UPDATE). Busy assigns a unique auto-incremented VchCode to every voucher and a unique Code to every master record. These serve as the deduplication keys in IIS.

Deduplication keys by data type:

- Sales, Purchases, Returns, Receipts: busy_vch_code (Tran1.VchCode — Busy's unique auto-generated voucher ID)
- Items / SKUs: busy_item_code (Item1.Code)
- Customers / Vendors: busy_account_code (Mast1.Code)
- Godowns / Branches: busy_godown_code
- Physical Stock: busy_vch_code + item_code + godown_code (composite key)

```
-- Example upsert pattern for sales
INSERT INTO sales (busy_vch_code, vch_no, vch_date, branch_id, sku_id, ...)
VALUES (...)
ON CONFLICT (busy_vch_code)
DO UPDATE SET
    qty = EXCLUDED.qty,
    rate = EXCLUDED.rate,
    updated_at = NOW();
```

> *DEV NOTE: Add a busy_vch_code column (BIGINT, UNIQUE) to the sales, purchases, sale_returns, purchase_returns, payment_receipts, and stock_transfers tables. Add a busy_item_code column (TEXT, UNIQUE) to skus. Add busy_account_code (TEXT, UNIQUE) to customers and vendors. These columns are the foreign keys linking IIS records back to Busy source records and are essential for correct upsert behaviour.*

4A.4 Busy SQL Queries for IIS Data

All data is retrieved via Service Code 1 (SQL query). The following are the canonical queries for each data type. The exact table and column names are Busy's internal schema — verify these against the client's Busy instance during initial integration setup.

4A.4.1 Item Master (SKUs)

```
-- Full fetch (no date filter for masters)
SELECT Code, Name, Alias, Unit, SalesRate, PurchaseRate,
       ItemGroup, HSNCode, TaxCategory
FROM Item1
ORDER BY Code
```

4A.4.2 Account Master (Customers and Vendors)

```
-- Customers: ParentGroup LIKE '%Sundry Debtors%'
SELECT Code, Name, Alias, Mobile, GSTNo, CreditLimit,
       Address1, Address2, ParentGroup
FROM Mast1
WHERE ParentGroup LIKE '%Sundry Debtors%'

-- Vendors: ParentGroup LIKE '%Sundry Creditors%'
SELECT Code, Name, Alias, Mobile, GSTNo,
       Address1, Address2, ParentGroup
FROM Mast1
WHERE ParentGroup LIKE '%Sundry Creditors%'
```

4A.4.3 Godowns / Warehouses

```
SELECT Code, Name, Address
FROM Godown1
ORDER BY Code
```

4A.4.4 Sales (VchType = 9)

```
-- Delta: add AND t.VchDate >= '{from_dt}' for incremental sync
SELECT t.VchCode, t.VchNo, t.VchDate, t.VchType,
       t.PartyName, t.GodownName, t.VchSeriesName,
       d.SrNo, d.ItemName, d.UnitName,
       d.Qty, d.Rate, d.Amount, d.STAmount,
       d.MC AS GodownDetail
FROM Tran1 t
JOIN Tran2 d ON t.VchCode = d.VchCode
WHERE t.VchType = 9
AND t.VchDate >= '{from_dt}'
AND t.VchDate <= '{to_dt}'
```

```
ORDER BY t.VchDate, t.VchCode
```

4A.4.5 Purchases (VchType = 2)

```
SELECT t.VchCode, t.VchNo, t.VchDate, t.VchType,
       t.PartyName, t.VchSeriesName,
       d.SrNo, d.ItemName, d.UnitName,
       d.Qty, d.Rate, d.Amount
FROM Tran1 t
JOIN Tran2 d ON t.VchCode = d.VchCode
WHERE t.VchType = 2
AND t.VchDate >= '{from_dt}'
AND t.VchDate <= '{to_dt}'
ORDER BY t.VchDate, t.VchCode
```

4A.4.6 Sale Returns (VchType = 3) and Purchase Returns (VchType = 10)

```
-- Same structure as Sales/Purchases query — just change VchType filter
WHERE t.VchType = 3   -- Sale Return
WHERE t.VchType = 10  -- Purchase Return
```

4A.4.7 Receipts — Customer Payments (VchType = 14)

Used when outstanding_method = 'computed' (Method B). Fetches payment receipts received from customers.

```
SELECT t.VchCode, t.VchNo, t.VchDate,
       t.PartyName, t.Narration,
       d.AccountName, d.AmtMainCur, d.AmountType
FROM Tran1 t
JOIN Tran3 d ON t.VchCode = d.VchCode
WHERE t.VchType = 14
AND t.VchDate >= '{from_dt}'
AND t.VchDate <= '{to_dt}'
```

4A.4.8 Physical Stock (VchType = 61)

Used to capture opening/closing stock snapshots. Physical Stock vouchers record actual counted stock quantities per item per godown.

```
SELECT t.VchCode, t.VchDate,
       d.ItemName, d.GodownName, d.Qty, d.Rate
FROM Tran1 t
JOIN Tran2 d ON t.VchCode = d.VchCode
WHERE t.VchType = 61
AND t.VchDate >= '{from_dt}'
ORDER BY t.VchDate DESC, d.ItemName
```

4A.4.9 Stock Transfers (VchType = 5)

```
SELECT t.VchCode, t.VchNo, t.VchDate,
       d.ItemName, d.Qty, d.Rate,
       d.FromGodown, d.ToGodown
FROM Tran1 t
```

```
JOIN Tran2 d ON t.VchCode = d.VchCode
WHERE t.VchType = 5
AND t.VchDate >= '{from_dt}'
AND t.VchDate <= '{to_dt}'
```

> *DEV NOTE: The exact column names in Tran1, Tran2, Tran3, Item1, Mast1, and Godown1 must be verified against the client's actual Busy instance using SC=1 with a 'SELECT TOP 1 * FROM TableName' query before going live. Column names may vary slightly by Busy version. Build a schema-discovery utility that IIS Super Admin can run during tenant onboarding to verify the column map.*

4A.5 Python Integration Layer

The Busy integration is implemented as a Python service module within the IIS backend. It handles HTTP communication with Busy, XML parsing, error handling, retry logic, and the sync scheduling.

4A.5.1 Core HTTP Client

```
import requests
import xml.etree.ElementTree as ET

BUSY_VCHTYPE_PURCHASE       = 2
BUSY_VCHTYPE_SALE_RETURN    = 3
BUSY_VCHTYPE_STOCK_TRANSFER = 5
BUSY_VCHTYPE_SALE           = 9
BUSY_VCHTYPE_PURCHASE_RETURN = 10
BUSY_VCHTYPE_RECEIPT        = 14
BUSY_VCHTYPE_PHYSICAL_STOCK = 61

BUSY_MASTERTYPE_ACCOUNT     = 2
BUSY_MASTERTYPE_ITEM        = 6
BUSY_MASTERTYPE_GODOWN      = 11
BUSY_MASTERTYPE_BRANCH      = 24

def call_busy_sql(busy_url, username, password, sql):
    headers = {
        'SC': '1',
        'Qry': sql,
        'UserName': username,
        'Pwd': password,
    }
    resp = requests.get(busy_url, headers=headers, timeout=60)
    result = resp.headers.get('Result', 'F')
    if result != 'T':
        error = resp.headers.get('Description', 'Unknown error')
        raise BusySyncError(f'Busy API error: {error}')
    return parse_busy_xml(resp.text)

def parse_busy_xml(xml_text):
```

```
    root = ET.fromstring(xml_text)
    rows = []
    for record in root.findall('Record'):
        row = {child.tag: child.text for child in record}
        rows.append(row)
    return rows
```

4A.5.2 Sync Scheduler

Use APScheduler (AsyncIOScheduler) integrated into the FastAPI application startup. Jobs are registered per tenant and stored in the database so they survive restarts.

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler

scheduler = AsyncIOScheduler()

# Register nightly transaction sync per tenant
scheduler.add_job(
    run_delta_sync,
    trigger='cron',
    hour=23, minute=0,
    args=[tenant_id, ['sales', 'purchases', 'receipts', 'stock_transfers']],
    id=f'delta_txn_{tenant_id}',
    replace_existing=True
)

# Register weekly master sync per tenant
scheduler.add_job(
    run_delta_sync,
    trigger='cron',
    day_of_week='sun', hour=1,
    args=[tenant_id, ['items', 'customers', 'vendors', 'godowns']],
    id=f'delta_master_{tenant_id}',
    replace_existing=True
)
```

4A.5.3 Error Handling and Retry

- If Busy is unreachable (connection refused, timeout): log status='failed' in sync_log, do NOT advance the from_dt watermark. Next scheduled run will retry from the same from_dt.
- If Busy returns Result='F': log the Description header as error_message in sync_log.
- Retry logic: 3 attempts with 30-second backoff before marking the sync as failed.
- Admin notification: if 3 consecutive nightly syncs fail, send an alert email to the Tenant Admin and to Super Admin.
- Manual retry: Tenant Admin can trigger a manual sync at any time from Settings → Busy Integration → Sync Now.

4A.6 Tenant Configuration — Busy Connection Settings

Each tenant that uses Busy integration must provide their Busy connection details. These are stored encrypted in the tenants table.

New columns added to the tenants table for Busy integration:

```
busy_integration_enabled   BOOLEAN DEFAULT FALSE,
busy_api_url               TEXT,     -- e.g. http://192.168.1.10:981
busy_username              TEXT,     -- Busy company username (encrypted at rest)
busy_password_enc          TEXT,     -- Busy company password (AES-256 encrypted)
busy_company_name          TEXT,     -- for display purposes
busy_last_full_sync_at     TIMESTAMPTZ,
busy_sync_enabled          BOOLEAN DEFAULT TRUE,
busy_sync_hour             INT DEFAULT 23  -- hour (0-23) for nightly sync
```

Settings page: Settings → Busy Integration

- Fields: Busy API URL, Username, Password (masked), Company Name
- 'Test Connection' button: runs SC=1 with 'SELECT 1' to verify connectivity and credentials
- 'Run Full Sync Now' button: triggers Phase 1 full sync as background job
- 'Sync Now' button: triggers immediate delta sync
- Sync schedule selector: choose nightly sync hour
- Sync history table: last 30 sync runs with status, records fetched, duration, errors

> *DEV NOTE: busy_password_enc must be AES-256 encrypted using a server-side key stored in environment variables (never in the database). Decrypt only at sync execution time, in memory, never log the decrypted password. This follows the same pattern as any credential stored in a SaaS platform.*

4A.7 Manual Upload as Fallback

The manual CSV/Excel upload module (Module 4) is retained as a fallback data ingestion method. It is used in three scenarios:

- Initial historical data load if Busy Web Service is not yet configured for the tenant.
- Gap-filling if the nightly sync failed for multiple nights and the tenant wants to manually catch up.
- Tenants who choose not to enable Busy Web Service integration (on-premise Busy without cloud access).

> *DEV NOTE: The data_ingestion_source column should be added to import_batches: TEXT, values 'busy_api' | 'manual_upload'. This allows the audit trail to show whether each batch came from the Busy sync or a manual upload. Analytics and reports must work correctly regardless of ingestion source — the underlying IIS data model is identical for both paths.*

4A.8 Integration Architecture & Known Limitations

This section documents how the Busy Web Service integration works at the network and operational level, and lists every known constraint.

4A.8.1 How the Integration Works

The Busy Web Service and IIS are two independent applications. No user needs to be logged in for the sync to run. The integration is server-to-server on a schedule.

38. Busy installs a Windows background service (busywebservice.exe) on port 981. The Busy desktop app does not need to be open.
39. IIS sends HTTP GET requests from the cloud server to http://{busy_ip}:981 nightly at 23:00. Authentication uses Busy credentials from the tenants table.
40. Busy returns XML. IIS parses and upserts into the tenant schema. No user action required.

> *DEV NOTE: The sync scheduler is a standalone Python process, independent of the web process. It reads tenant config directly from the database.*

4A.8.2 Known Limitations

L1 — Network Reachability (Critical). IIS cloud server must reach the Busy machine. Scenario A (recommended): same private cloud VPC, port 981 accessible by private IP. Scenario B: Busy on local office machine requires VPN or static public IP with port 981 forwarded.

> **IMPORTANT: Confirm which scenario applies before go-live. If Busy runs locally with no static IP or VPN the integration is NOT possible — fallback to manual file upload must be used.**

L2 — Busy Machine Must Be On. Must be powered on at 23:00 nightly. IIS retries once after 30 min and alerts after 3 consecutive failures.

L3 — Read-Only, No Write-Back. SC=1 is read-only. IIS never writes to Busy. Orders noted in IIS, MSL values, stock transfers, follow-ups, season tags are never reflected in Busy.

> *DEV NOTE: No SC=2 (write) integration is planned. All IIS-side planning data lives exclusively in the IIS tenant schema.*

L4 — Nightly Batch Only. IIS data reflects Busy as of the previous night. Manual delta sync available from Settings > Busy Sync.

L5 — Busy Edition. Web Service available in Busy Professional and Enterprise only. Starter/Basic must use manual file upload.

L6 — Internal Schema Dependency. IIS queries Busy internal tables (Tbltransaction, Tblallmasters etc.). Must re-validate after any Busy version upgrade.

L7 — Cost Encoding. Busy encodes purchase costs in a proprietary format. Tenant Admin must configure decode formula in Settings > Cost Decoder before profitability data is available.

L8 — No Pre-IIS History. One Full Sync on setup pulls all history in Busy (typically 2 years for Narang). Older data must be loaded via manual upload before Full Sync.

4A.8.3 Deployment Checklist

41. Confirm Busy edition supports Web Service (Professional or Enterprise)
42. Confirm network path and test connectivity to port 981 before go-live
43. Ensure Busy machine stays on overnight (no sleep/hibernate)
44. Store busy_ip, busy_username, busy_password in tenants table during onboarding
45. Run Test Connection from Settings > Busy Integration before enabling sync
46. Configure Cost Decoder before first profitability report
47. Run Full Sync immediately after onboarding
48. Verify sync_log shows success after first nightly run

5. Module: SKU Master & Season Tagging

5.1 User Stories

US-ID	As a…	I want to…	So that…
US-SKU-01	Tenant User	view a paginated, searchable list of all SKUs with key metrics	I can find and manage any SKU quickly

US-SKU-02	Tenant Admin	flag up to 300 SKUs as 'Focus SKUs'	the system gives them priority visibility across all reports
US-SKU-03	Tenant Admin	tag a SKU with one or more named seasonal windows (e.g. Monsoon: Jun–Sep)	the forecasting engine applies seasonal adjustments
US-SKU-04	Tenant Admin	bulk-tag multiple SKUs with the same season in one action	I don't have to tag 50 similar SKUs one by one
US-SKU-05	Tenant User	see a SKU's full profile: stock, DRR, WOI, season tags, profitability	I have all context in one place
US-SKU-06	System	preserve all user-defined flags and tags even after a new Busy import	my settings are never lost when I re-upload data

5.2 Pages

PAGE: SKU Master — List	
URL / Route	/skus
Access Roles	tenant_admin, tenant_user
Description	Paginated searchable table of all SKUs with inline metrics and action buttons.

Filters & Controls

Field Name	Type	Required	Validation / Notes	Source
search	Text	No	Searches sku_code and sku_name (ILIKE)	User input
filter_branch	Select	No	All Branches (Consolidated) | [Branch Name]. Defaults to active branch context.	User selection
filter_status	Select	No	All | Active | Inactive	User selection
filter_focus	Select	No	All | Focus SKUs only | Non-Focus	User selection
filter_season	Select	No	All | [List of season names]	User selection
filter_woi_status	Select	No	All | Red | Amber | Green	User selection

sort_by	Select	No	SKU Name | WOI | DRR | Stock | Margin %	User selection
page_size	Select	No	50 | 100 | 200	User selection

Table Columns

Column	Type	Sortable	Notes			
SKU Code	Text	Yes	Monospace font			
SKU Name	Text	Yes	Truncated at 40 chars with tooltip			
Category	Text	Yes				
Current Stock	Number	Yes	From latest inventory snapshot			
DRR (Rec)	Number	Yes	drr_recommended from forecasting_cache			
WOI	Number	Yes	Colour-coded: Red/Amber/Green			
Focus SKU	Checkbox	No	Toggle inline			
Season Tags	Badges	No	Shows all assigned seasons			
Margin %	Number	Yes	Hidden if profitability feature disabled			
Actions	Buttons	No	View | Tag Season			

PAGE: SKU Detail / Profile	
URL / Route	/skus/{sku_id}
Access Roles	tenant_admin, tenant_user
Description	Full profile page for a single SKU. Read-only for tenant_user; edit allowed for tenant_admin.

Sections on the Page

- Header: SKU Code, SKU Name, Category, Unit, Active/Inactive badge
- Inventory Panel: Current Stock, WOI (colour-coded), Last Import Date
- Forecasting Panel: DRR (4W / 13W / 52W / Recommended / Seasonal), Seasonal Uplift %, 12-Week Target, Suggested Order Qty
- MSL Panel: Busy MSL, System-Suggested MSL, Override field (admin only)

- Profitability Panel: Last Purchase Cost (decoded, admin only), Last Selling Price, Gross Margin %, Total Profit (13W)
- Season Tags Panel: Shows all tags; Add/Edit/Delete buttons (admin only)
- Sales History Chart: Bar chart of monthly sales, last 24 months
- Focus SKU Toggle: Checkbox (admin only)

PAGE: Season Tag Management	
URL / Route	/skus/{sku_id}/seasons
Access Roles	tenant_admin
Description	Add, edit, or remove season tags for a single SKU.

Field Name	Type	Required	Validation / Notes	Source
season_name	Text	Yes	Free text e.g. 'Monsoon', 'Summer', 'Pre-Winter'. Max 50 chars.	User input
start_month	Select (1–12)	Yes	Month number. 1=Jan, 12=Dec.	User selection
end_month	Select (1–12)	Yes	Must be >= start_month for same year, or system wraps (Dec→Jan allowed).	User selection

> *DEV NOTE: Season tags are stored as JSONB array in skus.season_tags: [{name:'Monsoon',start_month:6,end_month:9}]. Not a separate table — to avoid join complexity during forecasting.*

PAGE: Bulk Season Tagging	
URL / Route	/skus/bulk-tag
Access Roles	tenant_admin
Description	Assign a season tag to multiple selected SKUs in one action.

Workflow

49. User selects SKUs from a searchable/filterable list (same filters as SKU Master List).
50. User defines or selects an existing season tag (name + start_month + end_month).
51. On confirm: the season tag is appended to season_tags JSONB for each selected SKU. Existing tags are NOT replaced.
52. A summary shows: 'Season tag [Monsoon: Jun-Sep] applied to 47 SKUs.'

6. Module: Demand Forecasting Engine

6.1 User Stories

US-ID	As a…	I want to…	So that…
US-FC-01	Tenant User	see the Daily Run Rate (DRR) for each SKU across multiple time windows	I understand how fast each SKU is selling
US-FC-02	Tenant User	see Weeks of Inventory (WOI) with RAG colour coding for each SKU	I can instantly spot which SKUs need attention
US-FC-03	Tenant User	see a seasonally adjusted DRR for SKUs that have season tags	the forecast reflects real demand patterns, not just recent history
US-FC-04	Tenant User	see a Pre-Season Order Alert for SKUs whose season starts within the ordering horizon	I know exactly what to order before the season starts
US-FC-05	System	recompute all forecasting metrics within 30 seconds of a successful data import	the dashboard always shows fresh data

6.2 Business Logic — All Formulas

6.2.1 DRR Calculation

For each window, DRR = Total Units Sold in Window / Calendar Days in Window.

```
DRR_4w  = SUM(quantity) WHERE sale_date >= today-28  / 28
DRR_13w = SUM(quantity) WHERE sale_date >= today-91  / 91
DRR_52w = SUM(quantity) WHERE sale_date >= today-364 / 364
DRR_recommended = (DRR_4w * 0.5) + (DRR_13w * 0.35) + (DRR_52w * 0.15)
```

> *DEV NOTE: If a window period has fewer than 14 days of sales data, mark that DRR as NULL and exclude it from the weighted blend. Use the available windows only.*

6.2.2 Weeks of Inventory (WOI)

```
WOI = current_stock / (DRR_recommended * 7)
```

Colour coding:

- Red: WOI < woi_red_threshold (default: 4 weeks)
- Amber: WOI >= woi_red_threshold AND < woi_amber_threshold (default: 4–8 weeks)
- Green: WOI >= woi_amber_threshold (default: 8+ weeks)

> *DEV NOTE: All three thresholds are tenant-configurable in Settings → Inventory Targets. Never hardcode 4 or 8. Read woi_red_threshold and woi_amber_threshold from tenant settings on every WOI computation and display.*

> *DEV NOTE: If DRR_recommended is zero or NULL, WOI should be set to 999 and status 'green' (no demand = no risk). If stock is zero, WOI = 0 and status = 'red'.*

6.2.3 System-Suggested MSL

```
MSL_suggested = CEIL(DRR_recommended * lead_time_days * 1.2)
```

lead_time_days = configurable per tenant (default: 105 days = 3.5 months). The 1.2 multiplier adds 20% safety stock buffer.

6.2.4 12-Week Inventory Target

```
target_stock = CEIL(DRR_recommended * inventory_target_weeks * 7)
```

inventory_target_weeks = tenant setting (default: 12). Multiplying by 7 converts weeks to days. With default 12 weeks: 12 × 7 = 84 days of stock.

6.2.5 Suggested Order Quantity

```
suggested_order_qty = MAX(0, target_stock - current_stock)
```

6.2.6 Seasonal Demand Forecasting — 5-Step Engine

Step 1: Detect Active or Upcoming Seasonal Window

For each SKU, iterate over its season_tags array. For each tag:

53. Calculate the next occurrence of start_month relative to today.
54. If today is already within [start_month, end_month]: seasonal window is ACTIVE.
55. If the season starts within the next 140 days (20 weeks): window is UPCOMING.
56. If neither: skip seasonal logic for this tag.

Step 2: Calculate Seasonal Uplift from Prior-Year Data

For ACTIVE or UPCOMING seasonal windows:

```
Seasonal_DRR = SUM(quantity) WHERE sale_date in same calendar months, prior year /
days_in_that_period
Baseline_DRR = DRR_recommended (computed as above, from recent non-seasonal data)
Seasonal_Uplift_pct = ((Seasonal_DRR - Baseline_DRR) / Baseline_DRR) * 100
```

If 2+ years of prior data exist: average Seasonal_DRR across all available prior-year same-periods.

> *DEV NOTE: If Seasonal_DRR <= Baseline_DRR (no uplift or negative), log this but DO NOT apply a downward adjustment in Phase 1 — keep DRR_recommended as-is. Downward seasonal adjustments are a Phase 2 feature.*

Step 3: Apply Seasonal DRR to Forecast

When displaying the forecast for a SKU with an active/upcoming seasonal window:

- Show Baseline DRR and Seasonal DRR as separate values.
- Show Seasonal Uplift % clearly labelled.
- Use Seasonal_DRR (not Baseline) for ALL quantity calculations (WOI, target, suggested order) for weeks that fall within the seasonal window.

Step 4: Pre-Season Order Planning Alert

If a seasonal window is UPCOMING (starts within 20 weeks / 140 days):

```
season_duration_days = (end_month - start_month + 1) * 30  [approximate]
total_seasonal_demand = CEIL(Seasonal_DRR * season_duration_days)
stock_gap = MAX(0, total_seasonal_demand - current_stock)
latest_order_date = season_start_date - lead_time_days
```

The Pre-Season Order Alert Report (see Section 9) uses these computed values.

Set pre_season_alert = TRUE and latest_order_date in forecasting_cache for these SKUs.

Step 5: Post-Season Reversion

When today's date is after end_month of a season tag: if DRR_seasonal was being used, revert to DRR_recommended for all calculations. Clear the pre_season_alert flag.

6.3 Forecasting Computation Trigger

- Triggered automatically: on completion of any successful data import.
- Triggered automatically: on save of any stock transfer (recomputes affected SKU at both branches).
- Triggered manually: Admin can click 'Recompute Forecasts' button from the Settings page.
- Scheduled: nightly at 2 AM IST (cron job).
- Computation must complete in < 60 seconds for up to 10,000 SKUs across 20 branches.
- Each SKU-branch combination gets its own forecasting_cache row. A consolidated row (branch_id=NULL) is also computed per SKU.

> *DEV NOTE: Forecasting computation is O(SKUs × branches). For a tenant with 3,000 SKUs and 5 branches, that is 15,000 + 3,000 consolidated = 18,000 cache rows. This must run in parallel — use a job queue with branch-level parallelism.*

7. Module: Inventory Analytics & MSL Review

7.1 User Stories

US-ID	As a…	I want to…	So that…
US-INV-01	Tenant User	view a full inventory health report with WOI colour coding for all SKUs	I can see the overall stock situation at a glance
US-INV-02	Tenant User	filter the inventory view by WOI status (Red/Amber/Green), season tags, and branch	I can focus on the critical SKUs for a specific location

US-INV-03	Tenant Admin	see the system-suggested MSL alongside the Busy MSL for each SKU	I can see where reorder levels are set too low
US-INV-04	Tenant Admin	accept or override the system-suggested MSL for any SKU	I can correct suggestions that don't match business reality
US-INV-05	Tenant User	export the full inventory health report to Excel (with branch filter applied)	I can share a branch-specific report with the relevant manager or vendor

7.2 Pages

PAGE: Inventory Health Report	
URL / Route	/inventory
Access Roles	tenant_admin, tenant_user
Description	Full inventory health report. Default sort: WOI ascending (most critical first).

Summary Strip (top of page)

3 coloured boxes showing counts: Red SKUs (WOI < woi_red_threshold) | Amber SKUs (woi_red_threshold to woi_amber_threshold) | Green SKUs (WOI >= woi_amber_threshold). Clicking a box filters the table.

Table Columns

SKU Code	Name	Stock	DRR 4W	DRR 13W	DRR Rec	WOI	Status
AUTO-1042	Brake Pad Set — Alto 800	24	3.2	2.8	3.0	1.1w	RED
AUTO-1155	Clutch Plate — Wagon R	32	1.1	1.2	1.1	4.1w	AMBER
AUTO-4401	Spark Plug — i10	340	3.5	3.4	3.4	14.2w	GREEN

> *Example rows above show the visual pattern. Red rows have red-tinted WOI cell; amber rows amber-tinted.*

Additional Columns (visible on toggle)

- MSL (Busy) — from skus.msl
- MSL (System Suggested) — from forecasting_cache.msl_suggested
- MSL Override — editable field (admin only); stores in skus.msl_override
- Target Stock Quantity — from forecasting_cache.target_stock_qty (based on tenant's inventory_target_weeks setting)

- Suggested Order Qty — from forecasting_cache.suggested_order_qty
- Season — from skus.season_tags; shown as badge
- Seasonal DRR — from forecasting_cache.drr_seasonal (if applicable)

7.3 MSL Override Logic

When a Tenant Admin sets an MSL Override:

57. Store in skus.msl_override.
58. All reports and the dashboard must use msl_override when it is not NULL. Fall back to msl otherwise.
59. Show a visual indicator next to the MSL value when override is active (e.g. a pencil icon with tooltip 'Custom MSL set by admin').

7.4 Settings — Inventory Targets

PAGE: Inventory Targets	
URL / Route	/settings/inventory-targets
Access Roles	tenant_admin
Description	Configure the core planning parameters that drive WOI colour coding, stock targets, order quantities, and MSL calculations across the entire system. Changes take effect immediately across all pages and reports.

Field Name	Type	Required	Validation / Notes	Source
inventory_target_weeks	Number	Yes	Weeks of stock to hold at all times. Drives Target Stock Quantity and Suggested Order Quantity. Default: 12. Min: 4. Max: 52.	Tenant config
woi_red_threshold	Number	Yes	WOI below this value = Red (Critical). Must be < woi_amber_threshold. Default: 4 weeks.	Tenant config
woi_amber_threshold	Number	Yes	WOI below this = Amber. WOI ≥ this = Green (Healthy). Must be > woi_red_threshold. Default: 8 weeks.	Tenant config
lead_time_days	Number	Yes	Procurement lead time in days. Used in MSL suggestion and seasonal alert	Tenant config

			formulas. Default: 105 (3.5 months).	

60. A live preview panel updates as the admin types: 'With these settings — Red: WOI < [n]w | Amber: [n]–[n]w | Green: ≥[n]w | Target stock = [n] weeks | Pre-season alert window = lead time.'
61. When inventory_target_weeks is changed, target_stock_qty is recomputed for all SKUs in a background job. A status indicator on the Inventory Analytics and PO Advisor pages shows 'Updating targets…' until the job completes.
62. Changing woi_red_threshold or woi_amber_threshold immediately re-colours all WOI cells across all open sessions for this tenant.

> *DEV NOTE: These four values are stored in the tenants table (public schema). Read them in every query involving WOI thresholds, target stock, suggested order qty, and MSL suggestions. Never hardcode 4, 8, 12, or 105 anywhere in the application code.*

7B. Module: Smart Reorder Screen

The Smart Reorder Screen is a dedicated, always-available page that gives the user a consolidated actionable list of SKUs requiring attention. It replaces the need to manually scan the PO Advisor or inventory report. It runs two analysis buckets automatically and lets the user record order placement to suppress repeated alerts.

7B.1 MSL Rules (Updated)

MSL is maintained inside IIS only. It is not fetched from Busy. MSL is set at SKU + Branch + Godown level — meaning each combination of (SKU, Branch, Godown) can have its own MSL. The user sets MSL via the SKU Master or the Inventory Health Report.

- If msl = 0 for a (SKU, branch, godown) combination → that combination is excluded from all MSL alerts and the Smart Reorder Screen. Combinations with no MSL set (NULL) are also excluded.
- If msl > 0 → SKU is eligible for MSL-based analysis.
- effective_msl(sku, branch_id, godown_id) = sku_msl.msl — keyed on (sku_id, branch_id, godown_id). No Busy source. No override/fallback chain.

> *DEV NOTE: Remove msl_busy and msl_override columns from skus table. MSL is now stored in a separate sku_msl table (SKU + Branch + Godown granularity) rather than on the skus row. Schema: CREATE TABLE {tenant_schema}.sku_msl (id SERIAL PRIMARY KEY, sku_id UUID NOT NULL REFERENCES skus(id), branch_id UUID NOT NULL REFERENCES branches(id), godown_id UUID REFERENCES godowns(id), msl INT NOT NULL DEFAULT 0, UNIQUE(sku_id, branch_id, godown_id)). Update all MSL queries to JOIN sku_msl on (sku_id, branch_id, godown_id). The Smart Reorder Screen filters by the user's selected branch/godown context.*

7B.2 Two Analysis Buckets

7B.2.1 Bucket 1 — Last 7 Days Sold SKUs (Priority Analysis)

Identifies all SKUs that had at least one sale in the last 7 calendar days. For this subset only, runs a deep trend analysis.

Analysis steps for each Bucket 1 SKU:

- 1. Compute DRR over 4W, 13W, 52W windows using existing forecasting engine.
- 2. Check current effective_stock (at branch+godown level) vs sku_msl.msl for this (sku_id, branch_id, godown_id) combination.
- 3. Compute msl_suggested = CEIL(DRR_recommended * lead_time_days * 1.2) — same formula as existing.

- 4. If msl_suggested differs from sku_msl.msl (for this branch+godown) by more than 20%: flag as 'MSL Review Recommended'.
- 5. Compute suggested_order_qty using the confirmed formula: suggested_order_qty = MAX(0, (DRR_recommended × lead_time_days) + safety_buffer - effective_stock), where safety_buffer = DRR_recommended × (lead_time_days × 0.20). This is a trend-based projection plus a 20% safety buffer above lead-time demand, not a simple MSL deficit.
- 6. Apply seasonal uplift if season tag is active and season_start within lead_time_days.
- 7. Determine reorder_status (see Section 7B.4).

> *DEV NOTE: Bucket 1 query: SELECT DISTINCT sku_id FROM sales WHERE sale_date >= NOW() - INTERVAL '7 days' AND tenant_id = X. Join with skus WHERE msl > 0 OR msl IS NOT NULL (exclude msl=0/NULL). Run forecasting_cache refresh for these SKUs before displaying.*

7B.2.2 Bucket 2 — All SKUs (Broad Alert Scan)

Runs on all SKUs where msl > 0. Does not restrict to recent sales activity. Identifies SKUs at risk that may not have sold recently but still need attention.

Inclusion criteria for Bucket 2:

- effective_stock < sku_msl.msl for this (sku_id, branch_id, godown_id) combination
- effective_stock = 0 AND season_start within next lead_time_days (zero stock but season approaching)
- WOI < woi_red_threshold regardless of recent sales
- SKUs not already in Bucket 1 (no double-counting on screen)

> *DEV NOTE: Bucket 2 excludes any SKU already shown in Bucket 1 — the screen renders Bucket 1 first as a priority section, then Bucket 2 below as 'Other SKUs Needing Attention'. A user can collapse/expand each bucket independently.*

7B.3 Smart Reorder Page

URL / Route: /reorder

Access Roles: tenant_admin, tenant_user

Page refreshes analysis on load (or on manual 'Refresh Analysis' button). Shows two collapsible sections: Bucket 1 at the top, Bucket 2 below.

7B.3.1 Page Header

- Summary strip: count of SKUs in Bucket 1 | count in Bucket 2 | count with Order Noted status | count with MSL Review flags
- 'Refresh Analysis' button — re-runs both buckets on demand
- Branch filter — filter view to a specific branch or show consolidated

7B.3.2 Table Columns (both buckets)

Display columns per SKU row:

- SKU Code | SKU Name
- Current Stock
- MSL (user-set) — editable inline (admin only)
- System-Suggested MSL — from forecasting_cache.msl_suggested
- MSL Flag — 'MSL Review Recommended' badge if |msl_suggested - msl| > 20%. Clicking this badge opens the MSL Suggestion Prompt (see Section 7B.3.4).
- WOI
- Suggested Order Qty — from forecasting_cache.suggested_order_qty
- Basis — badge: 'Below MSL' | 'Out of Stock' | 'Seasonal Demand' | 'Trend Alert'

- Order Qty Input — editable number field (user enters qty they plan to order)
- Lead Time — toggle: 'Use system default (N days)' OR date picker for expected delivery date
- Action Button — 'Mark as Ordered' (saves order_noted record)
- Order Status — badge: 'Reorder Suggested' | 'Order Noted' | 'Pending Delivery' | 'No Action Required'
- Ordered Qty — shown once order noted
- Expected Delivery — shown once order noted

> *DEV NOTE: The Order Qty Input and Mark as Ordered button are always visible for eligible SKUs regardless of status. If a SKU already has Order Noted status, the button text changes to 'Update Order'. The row remains in the list so visibility is maintained — it just shows the order details instead of a blank input.*

7B.3.3 MSL Inline Edit

Admin users can edit the MSL directly from this screen. The MSL shown is for the currently active branch + godown context. Clicking the MSL cell makes it editable. Saving upserts into sku_msl immediately and triggers a re-evaluation of the SKU's status on the next analysis refresh.

> *DEV NOTE: MSL edits from the Reorder Screen and from the Inventory Health Report both write to the sku_msl table (INSERT ON CONFLICT UPDATE) keyed on (sku_id, branch_id, godown_id). No separate override table. The active branch + godown context determines which MSL row is shown and edited.*

7B.3.4 MSL Suggestion Prompt

When the user clicks the MSL Review Recommended badge on any SKU row, the system opens an inline prompt (modal or expandable panel within the row). This prompt presents the full MSL recommendation and asks the user to take an explicit action.

Prompt contents:

- Header: MSL Review Recommended for [SKU Code] — [SKU Name]
- Current MSL: [value from sku_msl] (set on [date] by [user])
- System Suggested MSL: [msl_suggested from forecasting_cache]
- Basis: e.g. Based on 13-week DRR of 2.4 units/day and lead time of 105 days with 20% safety buffer
- Trend note: e.g. Sales trend is increasing — DRR 4W (3.1) is higher than DRR 13W (2.4), suggesting demand is growing

User actions:

- Apply Suggestion — updates sku_msl.msl to msl_suggested. Records: applied_by (user), applied_at (timestamp), previous_msl (old value), new_msl (suggested value). Badge clears immediately.
- Keep Current — dismisses the prompt for this SKU for this analysis cycle. Records: dismissed_by, dismissed_at. Badge will reappear on next nightly analysis if the gap remains > 20%.

> *DEV NOTE: Store MSL suggestion decisions in a sku_msl_suggestions log table: (id, sku_id, branch_id, godown_id, old_msl, suggested_msl, decision [applied|dismissed], decided_by, decided_at). This gives admins a history of every MSL change decision. The prompt is ALWAYS triggered by user action (clicking the badge) — it never auto-applies without user confirmation.*

7B.4 Reorder Status Flow

Every eligible SKU on the Reorder Screen has one of the following statuses. Status is computed, not manually set (except 'Order Noted' which is triggered by user action).

Status values:

- No Action Required — stock >= msl and no trend alert
- Below MSL — effective_stock < skus.msl
- Out of Stock — effective_stock = 0
- Reorder Suggested — system recommends ordering based on trend/seasonality even if stock >= msl
- MSL Review Recommended — system-suggested MSL differs from user MSL by > 20%

- Order Noted — user has marked order as placed; suppresses daily alert
- Pending Delivery — order noted and expected delivery date is in the future
- Delivered / Stock Updated — expected delivery date has passed AND stock has increased (auto-detected from Busy sync)

> *DEV NOTE: Once a user marks 'Order Noted', the SKU's reorder_status changes to 'Order Noted'. The system will NOT re-generate a new alert for this SKU until: (a) the expected delivery date passes and stock has not increased, OR (b) the user manually clears the order record. This prevents daily duplicate alerts during long lead-time periods.*

7B.5 skus_reorder_orders Table (New)

Stores user-recorded order placements. One row per order placement action per SKU.

```
CREATE TABLE {tenant_schema}.skus_reorder_orders (
    id                   SERIAL PRIMARY KEY,
    sku_id               UUID NOT NULL REFERENCES skus(id),
    branch_id            UUID REFERENCES branches(id),  -- NULL = consolidated
    ordered_qty          INT NOT NULL,
    order_noted_at      TIMESTAMPTZ DEFAULT NOW(),
    placed_by            UUID REFERENCES users(id),
    use_system_lead_time BOOLEAN DEFAULT TRUE,
    expected_delivery_dt DATE,  -- NULL if use_system_lead_time=TRUE; computed on
save
    status               TEXT NOT NULL DEFAULT 'order_noted',
    -- 'order_noted' | 'pending_delivery' | 'delivered' | 'cancelled'
    notes                TEXT,
    created_at           TIMESTAMPTZ DEFAULT NOW(),
    updated_at           TIMESTAMPTZ DEFAULT NOW()
);
```

> *DEV NOTE: When use_system_lead_time = TRUE, set expected_delivery_dt = order_noted_at::DATE + tenant.lead_time_days at save time. When user picks a manual date, store that date directly. A background job runs nightly: for each order where status = 'pending_delivery' AND expected_delivery_dt < TODAY AND stock has not increased since order_noted_at, generate a 'delivery overdue' alert to the Tenant Admin.*

7B.6 API Endpoints — Smart Reorder

GET /reorder — Get both buckets for the current tenant

- Query params: branch_id (optional), refresh=true (forces re-run of analysis)
- Response: { bucket1: [...], bucket2: [...], summary: { b1_count, b2_count, ordered_count, msl_review_count } }

POST /reorder/orders — Mark order as placed

- Body: { sku_id, branch_id, ordered_qty, use_system_lead_time, expected_delivery_dt }
- Creates row in skus_reorder_orders, updates forecasting_cache.reorder_status

PATCH /reorder/orders/{id} — Update existing order record

- Body: { ordered_qty, expected_delivery_dt, status, notes }

GET /reorder/orders — List all open order records

- Returns all skus_reorder_orders where status IN ('order_noted', 'pending_delivery')

7B.7 User Stories — Smart Reorder Screen

- US-RO-01: As a Tenant User, I want to see all SKUs that need ordering in one place, so I can act without hunting across multiple reports.
- US-RO-02: As a Tenant User, I want to see separately: SKUs sold in the last 7 days vs all other at-risk SKUs, so I prioritise the most active items first.
- US-RO-03: As a Tenant User, I want to enter the quantity I've ordered and mark it as placed, so the system stops alerting me daily for the same SKU.
- US-RO-04: As a Tenant User, I want to set an expected delivery date (or use the system lead time), so the system knows when to follow up if stock hasn't arrived.
- US-RO-05: As a Tenant Admin, I want to edit the MSL for any SKU directly from this screen, so I can act on the system's MSL recommendation without navigating away.
- US-RO-06: As a Tenant User, I want the system to suggest if my MSL setting is too low or too high based on actual sales trend, so I can keep MSL values meaningful.
- US-RO-07: As a Tenant User, I want to see zero-stock SKUs flagged for upcoming seasonal demand, so I don't miss restocking before demand spikes.

7B.8 Acceptance Criteria — Smart Reorder Screen

- AC-RO-01: SKU+Branch+Godown combinations with no sku_msl row, or where sku_msl.msl = 0, do not appear on the Reorder Screen.
- AC-RO-02: Bucket 1 contains only SKUs with at least one sale in the last 7 days. Bucket 2 contains remaining at-risk SKUs not in Bucket 1.
- AC-RO-03: Marking 'Order Noted' for a SKU sets its status to 'Order Noted' and removes it from the alert queue. It remains visible in the list with order details.
- AC-RO-04: If use_system_lead_time = TRUE, expected_delivery_dt = order_noted_at + lead_time_days (rounded to date).
- AC-RO-05: If expected_delivery_dt has passed and effective_stock has not increased, status auto-updates to show delivery overdue alert.
- AC-RO-06: MSL Review Recommended badge appears when |msl_suggested - msl| / msl > 0.20. Clicking the badge opens the MSL Suggestion Prompt showing: (1) Current MSL, (2) System-Suggested MSL, (3) reason/basis for suggestion. User sees two options: 'Apply Suggestion' (updates sku_msl.msl to msl_suggested value) or 'Keep Current' (dismisses prompt and logs the decision with timestamp and user). Dismissal is per-SKU per analysis cycle — badge reappears on next nightly run if gap remains > 20%.
- AC-RO-07: Admin can edit msl inline for the active branch+godown context; change upserts to sku_msl; page re-evaluates status on next refresh.

8. Module: Purchase Order Advisor

8.1 User Stories

US-ID	As a…	I want to…	So that…
US-PO-01	Tenant User	generate a purchase order recommendation report at any time	I know exactly how much of each SKU to order
US-PO-02	Tenant User	switch between Volume-Only, Profit-Only, and Blended order modes	I can optimise orders based on my current business priority

US-PO-03	Tenant User	upload an Urgent SKU list and get a targeted order report for those SKUs	I can quickly respond to customer demand for out-of-stock items
US-PO-04	Tenant User	export any order report to Excel for sharing with vendors	I can send the order list directly
US-PO-05	Tenant User	see Seasonal Uplift applied in the order quantity for seasonal SKUs	I'm ordering the right quantity for the upcoming season

8.2 Pages

PAGE: PO Advisor — Regular Order	
URL / Route	/po-advisor
Access Roles	tenant_admin, tenant_user
Description	Generates order recommendations for all SKUs needing replenishment. Excludes SKUs with WOI > inventory_target_weeks (already adequately stocked) unless manually included.

Controls

Field Name	Type	Required	Validation / Notes	Source
branch_context	Select	Yes	Specific Branch | Consolidated (All Branches). When Consolidated: order qty is based on combined stock across all branches.	User selection
order_mode	Select	Yes	Volume-Only | Profit-Only | Blended (default)	User selection
lead_time_days	Number	Yes	Pre-filled from tenant settings (default 105). Editable per session.	Tenant config / user input
include_green_skus	Checkbox	No	Off by default. When on, includes WOI-Green SKUs in report.	User selection
filter_focus_only	Checkbox	No	Show Focus SKUs only	User selection
filter_season_alert	Checkbox	No	Show Pre-Season alert SKUs only	User selection

Report Table Columns

SKU Code	Name	Stock	DRR	WOI	13W Demand	12W Target	Order Qty	Priority

Row colour-coded by WOI status. Order Qty cell highlighted in blue.

Order Mode Logic

Mode	Sorting / Profitability Score Logic
Volume-Only	Sort by suggested_order_qty DESC. Profitability Score not shown.
Profit-Only	Sort by (gross_margin_pct * suggested_order_qty) DESC. Shows Profitability Score (1–5 stars based on gross_margin_pct quintile).
Blended (default)	Score = (0.6 * normalised_order_qty) + (0.4 * normalised_margin_pct). Sort by score DESC. Profitability Score shown.

PAGE: PO Advisor — Urgent SKU Report	
URL / Route	/po-advisor?tab=last-7-days&filter=urgent
Access Roles	tenant_admin, tenant_user
Description	Triggered after an Urgent SKU list upload. Displays order recommendations exclusively for the uploaded SKUs.

Table Columns (same as Regular PO plus)

- Urgency Source: 'Customer Requested' for all rows (from the upload)
- Priority: always URGENT badge
- Customer note if provided in the upload file

Export

Export to Excel includes all visible columns. Filename: Urgent_SKU_Order_{date}.xlsx.

9. Module: Seasonal Planning

9.1 User Stories

US-ID	As a…	I want to…	So that…
US-SEA-01	Tenant User	view a Pre-Season Order Alert report showing which seasonal SKUs need to be ordered before the season starts	I never miss the ordering window for seasonal stock

US-SEA-02	Tenant User	see the Latest Order Date for each seasonal SKU (season start minus lead time)	I know the exact deadline for placing an order
US-SEA-03	Tenant User	view a Seasonal SKU Calendar showing which SKUs peak in each month	I can plan purchasing cycles across the year
US-SEA-04	Tenant User	export the Pre-Season Order Alert report to Excel	I can share the seasonal order list with vendors

9.2 Pages

PAGE: Pre-Season Order Alert Report	
URL / Route	/seasonal/pre-season-alert
Access Roles	tenant_admin, tenant_user
Description	Automatically visible whenever any SKU has a season starting within the next 140 days (20 weeks). Branch-aware: shows stock gap using the active branch context (or consolidated).

Summary Banner (shown at top of page)

'X SKUs have seasons starting within the ordering horizon. Earliest order deadline: [date].'

Report Table Columns

SKU Code	Name	Season	Window	S.DRR	Total Demand	Stock	Gap to Order	Order By
AUTO-2301	Wiper Blade 18"	Monsoon	Jun–Sep	5.4	658	110	469	Apr 10

> *Gap to Order cell is red if gap > 0; green if current stock already covers total seasonal demand (gap = 0).*

Field Name	Type	Required	Validation / Notes	Source
SKU Code	Text	—	Monospace, links to SKU detail page	From skus table
SKU Name	Text	—		From skus table
Season Name	Text	—	From the season tag	From skus.season_tags
Window	Text	—	'Jun–Sep' format	Derived from season tag months
Seasonal DRR	Decimal	—	drr_seasonal from forecasting_cache	Computed

Total Seasonal Demand	Integer	—	Seasonal_DRR × season_duration_days	Computed
Current Stock	Integer	—	From latest inventory snapshot	From inventory_snapshots
Gap to Order	Integer	—	MAX(0, Total Demand - Current Stock). Red if > 0.	Computed
Latest Order Date	Date	—	Season start date - lead_time_days. Shown in red if within 14 days.	Computed

PAGE: Seasonal SKU Calendar	
URL / Route	/seasonal/calendar
Access Roles	tenant_admin, tenant_user
Description	Month-by-month visual showing which SKUs have seasonal peaks. On-demand view, always accessible.

Layout

12 rows (one per month). Each row shows: Month name | List of season tag names active in that month | Count of SKUs.

Clicking a month row expands to show the full list of SKUs with that season tag.

10. Module: Cost Decoder & Profitability Analytics

10.1 User Stories

US-ID	As a…	I want to…	So that…
US-PRF-01	Tenant Admin	configure the purchase cost decoding formula using character substitution and math operations	the system can decode the obfuscated cost codes from Busy
US-PRF-02	Tenant Admin	test a sample encoded value and see the decoded result before saving the formula	I can verify the formula is correct
US-PRF-03	Tenant Admin	view decoded purchase cost and gross margin for each SKU	I know the true profitability of my inventory
US-PRF-04	Tenant User	view a profitability ranking of SKUs by Gross Margin % and Total Gross Profit	I can see which SKUs are most profitable

US-PRF-05	Tenant User	see Volume-Profit Divergence alerts (high-volume SKUs with low margin)	I can identify SKUs that generate sales but erode profit

10.2 Cost Decoding Logic

10.2.1 Encoding Scheme

Busy stores purchase costs as encoded strings where each digit 0–9 is substituted with a character (e.g. A=1, B=2, C=3...). After substitution, a math operation may be applied (e.g. divide by 10).

Example: Encoded value 'ABC' with map A=1, B=2, C=3 and divide-by-10 → decoded cost = 12.3

10.2.2 Decoding Algorithm

```
1. For each character c in encoded_string:
   - If c is in char_map: replace with mapped digit
   - If c is a digit: keep as-is
   - If c is '.' or ',': treat as decimal separator
   - Otherwise: raise a decode error for this record
2. Concatenate to form a number string, convert to FLOAT
3. Apply math_operation:
   none     → result = number
   divide   → result = number / math_value
   multiply → result = number * math_value
   add      → result = number + math_value
   subtract → result = number - math_value
4. Round result to 2 decimal places
```

> *DEV NOTE: The decoding formula is stored per tenant in cost_decode_formulas. Always use the record where is_active=TRUE. Only one active formula per tenant at a time. When a new formula is saved, set the old one to is_active=FALSE and re-decode all historical purchase records.*

10.3 Pages

PAGE: Cost Decoder Configuration	
URL / Route	/settings/cost-decoder
Access Roles	tenant_admin
Description	Configure the purchase cost decoding formula. RESTRICTED — tenant_user cannot see this page.

Field Name	Type	Required	Validation / Notes	Source
char_map	Key-Value Editor	Yes	10 rows (0–9 mapped to characters). User	User input

			enters the character corresponding to each digit.	
math_operation	Select	Yes	None | Divide By | Multiply By | Add | Subtract	User selection
math_value	Number	Conditional	Required if math_operation != none. Decimal allowed.	User input
test_input	Text	No	Enter a sample encoded value to preview the decoded result. Live preview updates as user types.	User input / computed
test_output	Text (read-only)	—	Shows decoded result of test_input in real-time.	Computed

Save Behaviour

63. Validate: all 10 digit characters must be unique in the char_map.
64. Save new formula (is_active=TRUE), deactivate old formula.
65. Trigger background re-decoding of all purchase records and SKU purchase costs.
66. Show toast: 'Formula saved. Decoding all purchase records in background...'

PAGE: Profitability Analytics	
URL / Route	/profitability
Access Roles	tenant_admin (full view), tenant_user (limited — no purchase cost column)
Description	Profitability ranking and analysis.

Summary Cards (top row)

- Overall Gross Margin % (weighted by revenue, last 13 weeks)
- Total Gross Profit (last 13 weeks, in INR)
- Volume-Profit Divergence Count (# SKUs flagged)
- Best Margin SKU (name + margin %)

Tabs

Tab 1: By Margin % | Tab 2: By Total Profit | Tab 3: By Revenue | Tab 4: Divergence Alerts

Table Columns (Tabs 1, 2, 3)

#	SKU Code	Name	Margin %	Gross Profit	Revenue	Units Sold

Gross Margin Calculation

```
gross_profit_per_unit = last_selling_price - purchase_cost_decoded
gross_margin_pct = (gross_profit_per_unit / last_selling_price) * 100
total_gross_profit_13w = gross_profit_per_unit * units_sold_last_13w
```

> *DEV NOTE: purchase_cost_decoded is taken from the most recent purchase record for that SKU. If multiple purchase records exist, use the most recent rate_decoded.*

Volume-Profit Divergence Logic

A SKU is flagged as a divergence alert if:

```
rank_by_volume (last 13w) <= 20   AND   gross_margin_pct < MEDIAN(gross_margin_pct of
all SKUs)
```

The Divergence Alerts tab shows these SKUs with their volume rank, margin %, and an alert badge.

11. Module: Customer Outstanding Report

IIS supports two methods for tracking customer outstanding. The Tenant Admin selects the method that fits how their Busy data is available. Both methods produce the same report output — the difference is only in how the data enters the system.

	Method A — Direct Upload	Method B — Computed from Invoices & Receipts
How	Client exports the Busy Party Ledger / Outstanding report and uploads it to IIS. IIS reads debits and credits directly.	Client uploads Sales Invoices and Payment Receipts separately. IIS computes outstanding as invoices raised minus payments received.
Best when	Client already has a clean outstanding export from Busy and does not want to upload invoices and receipts separately.	Client uploads invoice-wise sales data anyway (which they do for forecasting) — adding payment receipts gives automatic outstanding computation with invoice-level drill-down.
Upload file	Busy Party Ledger / Outstanding export	Busy Payment Receipts export (Sales Invoices are already being uploaded)
Setting	outstanding_method = 'direct_upload' (default)	outstanding_method = 'computed'

11.1 User Stories

US-ID	As a…	I want to…	So that…
US-OUT-01	Tenant Admin	choose between Direct Upload and Computed as my outstanding method	I can use whichever method matches the Busy reports I have available

US-OUT-02	Tenant User	view a customer outstanding report with ageing buckets	I can see who owes money and for how long
US-OUT-03	Tenant User	drill into a customer's outstanding to see the individual transactions or invoices	I can see exactly what is outstanding before making a collection call
US-OUT-04	Tenant User	sort by total outstanding, filter by ageing bucket, search by customer name	I can prioritise collection calls
US-OUT-05	Tenant User	click 'Send WhatsApp' on any customer row to open a pre-filled payment reminder	I can reach out instantly without looking up the number
US-OUT-06	Tenant User	export the outstanding report to Excel and PDF	I can print it or share it externally
US-OUT-07	System (Auto)	recompute outstanding automatically whenever new outstanding ledger, payment receipt, or sales invoice data is imported	the report is always current after every upload

11.2 Method A — Direct Upload (outstanding_method = 'direct_upload')

11.2.1 Data Source

Reads from outstanding_ledger table. Each row is a debit (invoice / debit note) or credit (payment / credit note) against a customer. Uploaded from the Busy Party Ledger or Outstanding report.

11.2.2 Outstanding Computation

For each customer, compute net outstanding per ageing bucket using a running balance approach:

```
net_outstanding(customer) = SUM(positive amounts) - SUM(negative amounts)
                          = SUM(invoices + debit_notes) - SUM(payments + credit_notes)
```

Allocate remaining balance to ageing buckets based on transaction_date of each debit row, oldest-first:

```
Bucket 0-30d:    transaction_date >= today - 30
Bucket 31-60d:   transaction_date BETWEEN today-60 AND today-31
Bucket 61-90d:   transaction_date BETWEEN today-90 AND today-61
Bucket 91-180d:  transaction_date BETWEEN today-180 AND today-91
Bucket 180+d:    transaction_date < today - 180
```

> *DEV NOTE: Use the standard AR ageing approach — sum all debits minus credits per customer, then allocate remaining balance oldest-first across buckets. Do not bucket credits separately.*

11.3 Method B — Computed (outstanding_method = 'computed')

11.3.1 Data Source

Reads from sales_invoices (debits) and payment_receipts (credits). Sales invoices are already imported for forecasting — the only additional upload required is Payment Receipts.

11.3.2 Invoice-Level Outstanding

```
invoice_outstanding(invoice_no) =
  sales_invoices.invoice_total WHERE invoice_no = X
  - SUM(payment_receipts.amount_received WHERE invoice_no = X)
```

67. invoice_outstanding > 0: partially or fully unpaid.
68. invoice_outstanding = 0: fully settled — exclude from report.
69. invoice_outstanding < 0: overpayment — show as a credit balance, do not include in aged buckets.

11.3.3 Ageing Bucket Assignment

Bucket determined by invoice_date of each unpaid invoice:

```
Bucket 0-30d:    invoice_date >= today - 30
Bucket 31-60d:   invoice_date BETWEEN today-60 AND today-31
Bucket 61-90d:   invoice_date BETWEEN today-90 AND today-61
Bucket 91-180d:  invoice_date BETWEEN today-180 AND today-91
Bucket 180+d:    invoice_date < today - 180
```

> *DEV NOTE: Bucket assignment is per invoice, not per customer. Sum invoice_outstanding per bucket per customer for the customer's bucket totals.*

11.3.4 Customer Summary

```
customer_total_outstanding = SUM(invoice_outstanding) WHERE customer_id = X AND
invoice_outstanding > 0
last_invoice_date = MAX(invoice_date) for customer
last_payment_date = MAX(receipt_date) from payment_receipts for customer
```

11.4 Pages

PAGE: Outstanding Method Settings

11.4.2 outstanding_followups Table (New)

One row per follow-up action per invoice. Multiple follow-up rows per invoice are allowed (full history).

```
CREATE TABLE {tenant_schema}.outstanding_followups (
    id                    SERIAL PRIMARY KEY,
    invoice_ref           TEXT NOT NULL,  -- invoice_no from sales_invoices or
outstanding_ledger
    customer_id           UUID REFERENCES customers(id),
    comment               TEXT,           -- follow-up note from finance user
    promised_payment_dt   DATE,           -- date customer committed to pay
    snoozed_until         DATE,           -- suppress alert until this date
```

```
    followup_status       TEXT NOT NULL DEFAULT 'followup_pending',
    -- 'followup_pending' | 'customer_promised' | 'reminder_snoozed'
    -- | 'escalation_required' | 'auto_closed'
    created_by            UUID REFERENCES users(id),
    created_at            TIMESTAMPTZ DEFAULT NOW(),
    updated_at            TIMESTAMPTZ DEFAULT NOW()
);
```

> *DEV NOTE: outstanding_followups is invoice-level (keyed on invoice_ref), not customer-level. A customer may have multiple invoices, each with its own follow-up thread. The most recent row per invoice_ref is the active follow-up state. Older rows are history — never delete them.*

11.4.3 Follow-up Status Values

- followup_pending — no action taken yet; invoice is overdue and needs follow-up
- customer_promised — finance user has contacted customer; promised_payment_dt is set
- reminder_snoozed — user has snoozed the alert; system will not re-alert until snoozed_until date
- escalation_required — user has flagged this for escalation (admin review)
- auto_closed — payment has been received and matched from Busy sync; system set this automatically

> *DEV NOTE: There is NO 'received' or 'paid' status in outstanding_followups. The only way an invoice is considered paid is when the Busy sync imports a payment receipt that reduces the invoice_outstanding to 0 (Method B) or when the outstanding_ledger balance nets to 0 (Method A). The auto_closed status only means the follow-up thread is closed — the actual accounting close is driven entirely by Busy data.*

11.4.4 Enhanced Outstanding Report Page

The existing /outstanding page is enhanced with follow-up columns and inline actions per invoice row.

Additional columns added to the outstanding report table:

- Last Follow-up Comment — latest comment from outstanding_followups for this invoice
- Promised Payment Date — promised_payment_dt from latest follow-up
- Next Reminder Date — snoozed_until date if status = reminder_snoozed; else promised_payment_dt
- Follow-up Status — badge: 'Follow-up Pending' | 'Customer Promised' | 'Snoozed' | 'Escalation Required' | 'Auto Closed'

Action buttons per invoice row (inline):

- 'Add / Update Comment' — opens inline panel; fields: Comment (text), Promised Payment Date (date picker). Saves new row in outstanding_followups with status = 'customer_promised' if date set, else 'followup_pending'.
- 'Snooze' — opens snooze dialog: choose 1 day / 2 days / 7 days / custom date. Saves snoozed_until, sets status = 'reminder_snoozed'. Alert suppressed until snoozed_until.
- 'Escalate' — sets status = 'escalation_required'. Flags the invoice with an escalation badge. Notifies Tenant Admin by email.
- 'View History' — shows all outstanding_followups rows for this invoice in chronological order.

> *DEV NOTE: The snooze / escalate buttons replace the previously discussed 'Mark as Received' button. There is NO 'Received' button on the outstanding follow-up UI. Payment is auto-detected from Busy sync only. This is a hard requirement to avoid confusion between operational follow-up closure and actual accounting receipt.*

11.4.5 Auto-Close Logic

When the Busy nightly sync imports new payment data, the system checks all open outstanding_followups records:

- Method B (computed): after importing payment_receipts, recompute invoice_outstanding for each invoice_ref in outstanding_followups. If invoice_outstanding = 0: insert new outstanding_followups row with status = 'auto_closed'.
- Method A (direct upload): after importing outstanding_ledger, recompute net balance per party. If net balance for a customer reaches 0 or goes negative (overpayment): mark all their open follow-ups as 'auto_closed'.

DEV NOTE: Auto-close inserts a new row (it does not update the existing row). This preserves the full history. The active status is always determined by the most recent row per invoice_ref.

11.4.6 Missed Payment Alert Logic

A background job runs nightly after the Busy sync completes:

- For each outstanding_followups row where followup_status = 'customer_promised' AND promised_payment_dt < TODAY AND invoice is still unpaid: generate an in-app alert and email to the Finance User who recorded the follow-up.
- Alert message: 'Payment committed by [Customer Name] for invoice [Invoice No] was due on [Date] but has not been received. Please follow up.'
- For rows where followup_status = 'reminder_snoozed' AND snoozed_until <= TODAY: revert status to 'followup_pending' so the invoice re-appears in the alert queue.

DEV NOTE: Missed payment alerts are generated once per day per invoice, not repeatedly throughout the day. Once the Finance User takes any action (snooze, update comment, escalate), the alert is cleared for that day.

11.4.7 Acceptance Criteria — Follow-up

- AC-OUT-08: Finance user can add a comment + promised payment date against any outstanding invoice. The action creates a new row in outstanding_followups.
- AC-OUT-09: If promised_payment_dt has passed and invoice is still unpaid, the system generates a missed-payment alert the following morning.
- AC-OUT-10: Snoozing an invoice suppresses its alert until snoozed_until date. After that date, status reverts to followup_pending.
- AC-OUT-11: When Busy sync detects full payment for an invoice, the system inserts an auto_closed follow-up row. The invoice disappears from the active follow-up queue.
- AC-OUT-12: There is no 'Received' button in the UI. Payment status is determined by Busy data only.
- AC-OUT-13: All follow-up rows are preserved for audit. View History shows complete chronological log per invoice.

URL / Route	/settings/outstanding
Access Roles	tenant_admin
Description	Configure which outstanding method is active for this tenant.

Field Name	Type	Required	Validation / Notes	Source
outstanding_method	Radio	Yes	Direct Upload: upload Busy outstanding/ledger export. Computed: upload Payment Receipts — IIS calculates outstanding from invoices and receipts.	User selection

70. Changing the method takes effect immediately for the report. Existing data in outstanding_ledger and payment_receipts is retained — switching back restores the previous state.
71. A banner is shown on the outstanding report and import page indicating which method is currently active.

PAGE: Customer Outstanding Report	
URL / Route	/outstanding
Access Roles	tenant_admin, tenant_user
Description	Ageing report. Data source depends on outstanding_method. Both methods produce the same report layout. Refreshed automatically after each relevant import.

A banner at the top of the page shows: 'Outstanding data source: Direct Upload (Busy ledger)' or 'Outstanding data source: Computed from invoices & receipts'. Clicking the banner links to Settings → Outstanding Method.

Summary Strip

5 coloured tiles: 0–30d (green), 31–60d (blue), 61–90d (amber), 91–180d (red), 180+d (dark red). Plus Total Outstanding tile.

Customer Summary Table

Customer Name	Total O/S	0–30d	31–60d	61–90d	91–180d	180+d	

- Additional columns: Last Invoice / Transaction Date | Last Payment Date | Credit Balance (if any) | WhatsApp button
- Row click expands to transaction-level detail (Method A: ledger rows; Method B: invoice-level drill-down with Paid So Far and Balance Due columns).

12. Module: WhatsApp Integration

12.1 User Stories

US-ID	As a…	I want to…	So that…
US-WA-01	Tenant User	click a WhatsApp button next to a customer and have WhatsApp Web open with the customer's number pre-loaded	I can send a message without copying numbers manually
US-WA-02	Tenant User	select a pre-configured message template before sending	I send consistent, professional messages

US-WA-03	Tenant Admin	create, edit, and delete message templates with dynamic variables	I can customise message content for different situations

12.2 Technical Implementation

12.2.1 WhatsApp URL Scheme

```
https://wa.me/{phone_number}?text={url_encoded_message}
```

phone_number: customer.whatsapp_number if set, else customer.phone. Must be in international format without + (e.g. 919876543210 for India).

On click: open URL in a new browser tab. No API key required. Works on all devices.

12.2.2 Phone Number Formatting

```
1. Strip all non-numeric characters from stored phone number
2. If number starts with 0: replace leading 0 with country code (91 for India)
3. If number is 10 digits: prepend 91
4. Result must be 12 digits for India (91 + 10-digit mobile)
```

> *DEV NOTE: Store phone number validation configuration per tenant (country code default: 91). Phase 1 assumes all customers are Indian mobile numbers.*

12.2.3 Template Variable Substitution

Supported variables and their data sources:

Variable	Replacement Value	Source
{customer_name}	customers.customer_name	customers table
{outstanding_amount}	Formatted as ₹X,XX,XXX (Indian number format)	Computed from ledger
{due_date}	Date of oldest outstanding invoice	Computed from ledger
{shop_name}	tenants.business_name	tenants table

12.3 Pages

PAGE: WhatsApp Templates	
URL / Route	/settings/whatsapp-templates
Access Roles	tenant_admin
Description	Manage message templates.

Field Name	Type	Required	Validation / Notes	Source
template_name	Text	Yes	Max 100 chars. Must be unique within tenant.	User input
message_body	Textarea	Yes	Max 1000 chars. Supports variables listed above. Live preview shown below the field.	User input
is_default	Checkbox	No	Only one template can be default. Saving a new default unchecks the previous one.	User selection

13. Module: Tenant Dashboard

13.1 User Stories

US-ID	As a…	I want to…	So that…
US-DASH-01	Tenant User	see a real-time business overview dashboard on login	I have an immediate picture of inventory health and business status
US-DASH-02	Tenant User	see critical WOI-Red SKUs prominently on the dashboard	I take action on stock-outs immediately
US-DASH-03	Tenant User	see Pre-Season Alerts on the dashboard when seasonal ordering deadlines are approaching	I never miss a seasonal buying window
US-DASH-04	Tenant User	see customer outstanding summary on the dashboard	I stay on top of receivables without navigating away

13.2 Page Spec

PAGE: Tenant Dashboard	
URL / Route	/dashboard
Access Roles	tenant_admin, tenant_user
Description	Main landing page post-login. A Branch Selector in the global header controls the context for all widgets. Selecting 'All Branches (Consolidated)' aggregates data

	across the entire tenant. All data sourced from pre-computed caches — no live computation on page load. Must load in < 5 seconds.

13.3 Widget Specifications

#	Widget Name	Data Source	Display	Click Action
1	Branch Selector (Global)	branches table	Dropdown in header: lists all active branches + 'All Branches (Consolidated)'. Persists for session.	—
2	Inventory Health RAG Summary	forecasting_cache WHERE branch_id = active branch (or NULL for consolidated)	3 pills: Red N / Amber N / Green N	Navigate to /inventory filtered by status
3	Top 10 Stock-Out Risk SKUs	forecasting_cache ORDER BY woi ASC LIMIT 10	Table: SKU, Stock, DRR, WOI, Branch	Navigate to SKU detail
4	Sales Trend Chart	sales GROUP BY week, last 24 weeks, scoped to branch	Bar chart	—
5	DRR Trend (Top SKUs)	forecasting_cache, sales history, scoped to branch	Line chart for top 5 Focus SKUs	—
6	Customer Outstanding Summary	computed ageing totals (tenant-level, no branch filter)	5 bucket tiles + total + top 5 overdue	Navigate to /outstanding
7	Focus SKU Performance	skus WHERE is_focus_sku=TRUE, forecasting_cache, branch-scoped	Table: Stock vs MSL vs WOI	Navigate to /inventory
8	Top 300 by Sales	sales aggregated by sku, last 52w, branch-scoped	Table: SKU, Revenue, Units	Navigate to SKU detail
9	Pending Orders Status	manual entry (Phase 1: informational only)	Text card	—
10	Urgent SKU Status	import_batches WHERE data_type=urgent_skus	Last upload date + count	Navigate to /po-advisor?tab=last-7-days&filter=urgent
11	Top 10 by Gross Profit	profitability view, branch-scoped	Bar chart / table	Navigate to /profitability

12	Top 10 by Margin %	profitability view, branch-scoped	Horizontal bar chart	Navigate to /profitability
13	Volume-Profit Divergence	Divergence computation, branch-scoped	Alert card with count + top 3	Navigate to /profitability
14	Gross Margin Trend	Sales + purchase costs, weekly, branch-scoped	Line chart, last 12 weeks	—
15	Pre-Season Order Alert	forecasting_cache WHERE pre_season_alert=TRUE, branch-scoped	Alert card: season names + deadlines + count	Navigate to /seasonal/pre-season-alert
16	Seasonal SKU Calendar	skus.season_tags aggregated by month	12-cell month grid with season counts	Navigate to /seasonal/calendar
17	Data Freshness Indicator	import_batches latest by data_type AND branch	Table: branch + data type + last import date	Navigate to /import/history
18	Cross-Branch Stock Alert	forecasting_cache: SKUs where any branch = Red but another branch = Green	Alert count + top 3 SKUs with transferable stock. Clicking opens Cross-Branch Comparison → Inventory tab filtered to those SKUs.	Navigate to /branches/comparison?tab=inventory
19	Branch Sales Snapshot	sales aggregated by branch_id, last 13 weeks	Side-by-side bar chart: total revenue per branch. Shows % share of total revenue per branch. Visible only when 2+ branches exist.	Navigate to /branches/comparison?tab=sales

14. Super Admin Portal

The Super Admin portal is a completely separate web application / route namespace. It is accessible ONLY at admin.iis.in (or /admin/* with its own auth middleware). No tenant user can reach any Super Admin route.

14.1 User Stories

US-ID	As a…	I want to…	So that…
US-SA-01	Super Admin	view a real-time platform overview with all tenants, MRR, and health metrics	I have a live pulse on the entire platform
US-SA-02	Super Admin	create a new tenant and have their data environment auto-provisioned	I can onboard clients in under 60 seconds
US-SA-03	Super Admin	suspend or reactivate a tenant account	I can manage non-paying or problematic clients
US-SA-04	Super Admin	define subscription plans with configurable parameters	I can offer different tiers to different clients
US-SA-05	Super Admin	view and reset any user account across all tenants	I can help a tenant admin who is locked out
US-SA-06	Super Admin	broadcast a maintenance announcement to all tenant portals	tenants are informed of downtime before it happens
US-SA-07	Super Admin	view a full audit log of all Super Admin actions	all platform admin activity is traceable

14.2 Pages

PAGE: Super Admin — Overview Dashboard	
URL / Route	/admin/dashboard
Access Roles	super_admin
Description	Platform-level KPI overview. Data sourced from the public schema.

KPI Cards

- Total Tenants (count by status)
- Monthly Recurring Revenue (sum of plan prices for active tenants)
- Trials Expiring in 7 Days (count)
- Platform Uptime % (last 30 days — from a health_checks log table)
- Active Sessions Now (count of JWT tokens issued in last 30 minutes)
- Failed Import Jobs Today (cross-tenant count from import_batches)

Tenant Table (same as /admin/tenants, top 7 rows)

- Business Name | Plan | Status | Users | SKUs | Last Login | MRR Contribution | Joined

Other Widgets

- Plan Distribution bar chart
- Tenant Growth bar chart (new tenants per month, last 12 months)
- Activity Feed (last 10 cross-tenant audit events)
- Active Announcement banner

PAGE: Tenant Management	
URL / Route	/admin/tenants
Access Roles	super_admin
Description	Full list of all tenants. Create, view, edit, suspend, reactivate, delete.

Table Filters

- Status: All | Active | Trial | Suspended | Churned

Table Columns

Business Name	Plan	Status	Users	SKUs	Last Login	Revenue	Joined	Actions

Create New Tenant — Form Fields

Field Name	Type	Required	Validation / Notes	Source
business_name	Text	Yes	Max 200 chars	User input
contact_name	Text	Yes	Max 100 chars	User input
email	Email	Yes	Must be unique across tenants. This becomes the Tenant Admin login email.	User input
phone	Text	No		User input
plan_id	Select	Yes	Dropdown of active plans	User selection
trial_days	Number	No	If set, tenant starts on trial for N days. Leave blank for direct paid plan.	User input

Create Tenant — Backend Actions (must all happen atomically)

72. Create tenants record in public schema.
73. Create PostgreSQL schema: tenant_{uuid}.

74. Run database migrations to create all required tables in new schema.
75. Create tenant_users record with role=tenant_admin for the email provided.
76. Generate temporary password, send onboarding email with login URL and credentials.
77. Log action to audit_log.
78. Entire provisioning flow must complete in < 60 seconds.

PAGE: Subscription Plan Manager

URL / Route	/admin/plans
Access Roles	super_admin
Description	Define and edit subscription plans.

Field Name	Type	Required	Validation / Notes	Source
name	Text	Yes	Plan name e.g. Starter, Growth, Pro	User input
price_monthly	Number	Yes	INR	User input
price_annual	Number	No	INR. Leave blank if annual billing not offered.	User input
max_users	Number	Yes	Max users per tenant on this plan	User input
max_skus	Number	Yes	Max SKUs per tenant	User input
retention_months	Number	Yes	Data retention period in months	User input
feature_profitability	Toggle	Yes	Enable/disable Profitability Analytics module for this plan	User toggle
feature_whatsapp	Toggle	Yes	Enable/disable WhatsApp Integration for this plan	User toggle
is_active	Toggle	Yes	Inactive plans cannot be assigned to new tenants	User toggle

PAGE: All Users (Cross-Tenant)

URL / Route	/admin/users
Access Roles	super_admin
Description	Searchable list of every user across all tenants.

Table Columns

- User Name | Email | Tenant Name | Role | Status (active/disabled) | Last Login | Actions

Actions per row

- Reset Password: generates new temporary password, emails user
- Disable / Re-enable Account
- View Login Activity: modal showing last 20 login timestamps and IPs

PAGE: Platform Health

URL / Route	/admin/health
Access Roles	super_admin
Description	Real-time system health and error monitoring.

Panels

- KPI bar: Uptime % | Active Sessions | Avg API Response (ms) | DB Storage Used (GB)
- Storage by Tenant: horizontal bar chart
- Error Log: table of system errors and failed jobs, filterable by tenant and date range
- Import Jobs in Progress: live list with tenant name, file name, % progress

PAGE: Announcements

URL / Route	/admin/announcements
Access Roles	super_admin
Description	Create and manage in-app announcements broadcast to tenant portals.

Field Name	Type	Required	Validation / Notes	Source
title	Text	Yes	Max 200 chars	User input
body	Textarea	Yes	Max 1000 chars. Markdown supported.	User input
target	Select	Yes	All Tenants | Specific Tenant (with search)	User selection
display_from	DateTime	Yes	When to start showing the announcement	User input
display_until	DateTime	Yes	When to stop showing	User input
type	Select	Yes	info | warning | maintenance	User selection

Active announcements appear as a dismissable banner at the top of the tenant portal on next login.

PAGE: Audit Log

URL / Route	/admin/audit-log
Access Roles	super_admin
Description	Immutable log of all Super Admin actions.

Table Columns

- Timestamp | Action Type | Actor (Super Admin name) | Target (Tenant / User / Plan) | Details | IP Address

Actions logged

- tenant_created | tenant_suspended | tenant_reactivated | tenant_deleted
- plan_assigned | plan_created | plan_updated
- user_password_reset | user_disabled | user_enabled
- announcement_sent | formula_viewed (cross-tenant)

Export

- Export to CSV with date range filter

15. Reports Reference

All downloadable reports follow these standards unless specified otherwise:

- Format: Excel (.xlsx) via a server-side export endpoint. PDF available for Outstanding Report only.
- Filename convention: ReportName_TenantSlug_YYYY-MM-DD.xlsx
- Export endpoint: GET /api/reports/{report_id}/export?format=xlsx
- Reports are generated on-demand; no scheduled email delivery in Phase 1.

Report Name	Trigger	Formats	Key Fields
Purchase Order Recommendation	On-demand	Excel	SKU, Stock (branch or consolidated), DRR, WOI, Order Qty, Priority, Profit Score
Urgent SKU Order Report	After urgent SKU upload	Excel	SKU, Stock, DRR, Order Qty, Priority=URGENT
MSL Review Report	On-demand	Excel	SKU, Busy MSL, System MSL, Variance, Recommendation
Customer Outstanding Report	On-demand / Auto-refresh on import	Excel + PDF	Customer, Total O/S, 0-30d, 31-60d, 61-90d, 91-180d, 180+d, Last Invoice Date, Last Payment Date. Drill-down: Invoice No, Invoice Date, Days O/S, Invoice Total, Paid, Balance Due
Inventory WOI Report	On-demand	Excel	SKU, Branch (or Consolidated), Stock, DRR, WOI, Status, Season
Top 300 by Sales	Auto on import	Excel	Rank, SKU, Units Sold, Revenue, WOI

Focus SKU Report	On-demand	Excel	SKU, Stock, DRR, WOI, MSL, Season, Margin%
Sales Forecast Report	On-demand	Excel	SKU, DRR (4W/13W/Rec/Seasonal), Projected 13W Sales, Season Flag
Pre-Season Order Alert	Auto when season within 20 weeks	Excel	SKU, Season, Window, S.DRR, Total Demand, Stock (branch/consolidated), Gap, Order By Date
Seasonal SKU Calendar	On-demand	Screen only	Month × Season matrix with SKU counts
Profitability Report	On-demand / After import	Excel	SKU, Margin%, Gross Profit, Revenue, Units Sold, Branch, Divergence Flag
Volume-Profit Divergence	On-demand	Excel	SKU, Volume Rank, Margin%, Alert reason
Stock Transfer Log	On-demand	Excel	Date, SKU, From Branch, To Branch, Qty, Transferred By, Notes
Cross-Branch Comparison — Inventory	On-demand	Excel	SKU, [Per-Branch: Stock / DRR / WOI], Consolidated Stock, Transfer Opportunity Flag
Cross-Branch Comparison — Sales	On-demand	Excel	SKU, [Per-Branch: Units Sold / Revenue / Share %], Total Units, Total Revenue, Highest-Sales Branch
Cross-Branch Comparison — Profitability	On-demand (Admin only)	Excel	SKU, [Per-Branch: Margin % / Gross Profit / Revenue], Best Margin Branch, Divergence Flag
Cross-Branch Top SKUs	On-demand	Excel	One sheet per branch: Rank / SKU / Units / Revenue / Margin %. Common Top SKU flag.

16. API Contracts (Key Endpoints)

Base URL: https://api.iis.in/v1. All tenant-scoped endpoints require: Authorization: Bearer {JWT} in headers. JWT payload must include: tenant_id, user_id, role.

> *DEV NOTE: The middleware must validate that the tenant_id in the JWT matches the schema being queried. Any mismatch must return 403 Forbidden.*

16.1 Authentication

Method	Endpoint	Auth Required	Description
POST	/auth/login	No	Body: {email, password}. Returns: {access_token, refresh_token, user, tenant}
POST	/auth/refresh	Refresh token	Rotate access token

POST	/auth/logout	Yes	Invalidate refresh token

16.2 Data Import

Method	Endpoint	Auth	Description
POST	/imports/upload	tenant_user+	Multipart form: file + data_type + notes. Returns import_batch_id.
GET	/imports	tenant_user+	List import batches for tenant. Query: ?page&limit&status&data_type
GET	/imports/{id}	tenant_user+	Get status + error log of one import batch

16.3 SKU & Forecasting

Method	Endpoint	Auth	Description
GET	/skus	tenant_user+	Paginated SKU list with forecast data. Query: ?search&woi_status&season&focus&sort&page&limit&branch_id (omit for consolidated)
GET	/skus/{id}	tenant_user+	Full SKU profile including forecasting_cache
PATCH	/skus/{id}	tenant_admin	Update: is_focus_sku, msl_override, season_tags
POST	/skus/bulk-tag	tenant_admin	Body: {sku_ids[], season_tag{name,start_month,end_month}}
GET	/forecasting/pre-season-alerts	tenant_user+	All SKUs with pre_season_alert=TRUE
POST	/forecasting/recompute	tenant_admin	Manually trigger forecasting recomputation for tenant

16.4 Reports

Method	Endpoint	Auth	Description
GET	/reports/po-recommendation	tenant_user+	Query: ?mode=blended|volume|profit&lead_time_days&branch_id (omit for consolidated)
GET	/reports/po-recommendation/export	tenant_user+	Returns .xlsx file download
GET	/reports/inventory-woi	tenant_user+	Full inventory WOI report. Query: ?branch_id (omit for consolidated)

GET	/reports/outstanding	tenant_user +	Customer outstanding with ageing (tenant-level, no branch filter)
GET	/reports/outstanding/export	tenant_user +	Query: ?format=xlsx|pdf
GET	/reports/profitability	tenant_admin	Query: ?sort=margin|profit|revenue&period=13w|26w|52w&branch_id
GET	/reports/pre-season-alert	tenant_user +	Query: ?branch_id (omit for consolidated)

16.6 Branches

Method	Endpoint	Auth	Description
GET	/branches	tenant_user +	List all branches for tenant. Query: ?include_inactive=true
POST	/branches	tenant_admin	Create new branch. Body: {branch_name, branch_code, address}
PATCH	/branches/{id}	tenant_admin	Update name, address, active status, or set as home branch
DELETE	/branches/{id}	tenant_admin	Only permitted if no import data exists for this branch. Returns 409 if data exists — must deactivate instead.
GET	/branches/comparison	tenant_user +	Cross-branch comparison. Query: ?tab=inventory|sales|profitability|top-skus &sku_ids[]&period=4w|13w|26w|52w&woi_status&top_n. Note: tab=profitability returns 403 for tenant_user role.
GET	/branches/comparison/export	tenant_user +	Export active tab to Excel. Query: ?tab=inventory|sales|profitability|top-skus &[same filters as above]. Returns .xlsx file.
POST	/branches/transfer	tenant_admin	Record a stock transfer. Body: {transfer_date, sku_id, from_branch_id, to_branch_id, quantity, notes}
GET	/branches/transfer/log	tenant_user +	Transfer log. Query: ?branch_id&sku_id&from&to&page&limit
GET	/branches/column-maps	tenant_admin	List saved Sale From column mappings for this tenant (one per data_type)
POST	/branches/column-maps	tenant_admin	Confirm a Sale From column mapping. Body: {data_type, column_name}. Saves to branch_column_maps.
DELETE	/branches/column-maps/{id}	tenant_admin	Remove a saved mapping — system will prompt again on next upload of that data type.

16.5 Super Admin

Method	Endpoint	Auth	Description
GET	/admin/tenants	super_admin	List all tenants
POST	/admin/tenants	super_admin	Create + provision new tenant
PATCH	/admin/tenants/{id}	super_admin	Update profile, plan, status
DELETE	/admin/tenants/{id}	super_admin	Permanently delete tenant + schema (irreversible)
GET	/admin/users	super_admin	All users across all tenants
POST	/admin/users/{id}/reset-password	super_admin	Generate and email new temp password
GET	/admin/audit-log	super_admin	Query: ?from&to&action_type&page
POST	/admin/announcements	super_admin	Create new announcement

17. Non-Functional Requirements

Category	Requirement	Metric / Target	Priority
Multi-Tenancy	Complete data isolation — no cross-tenant data leak possible	Zero tolerance — tested via automated query interception tests	P0 — Critical
Tenant Provisioning	New tenant schema provisioned on creation (includes Home Branch)	< 60 seconds end-to-end	P0 — Critical
Dashboard Load Time	Tenant dashboard loads from cache	< 5 seconds (P95)	P1 — High
Forecasting Compute	Full recompute for all SKUs × all branches + consolidated rows	< 60 seconds for 10,000 SKUs × 20 branches	P1 — High
API Response Time	Standard read API endpoints	< 500ms (P95)	P1 — High
File Import	File parsing job picks up within	< 10 seconds of upload	P1 — High
Availability	Platform uptime during business hours	99.5% (9 AM–8 PM IST)	P1 — High
Scalability	Concurrent tenants, SKUs, and branches	100 tenants × 10,000 SKUs × 20 branches each	P2 — Medium
Security	Authentication	JWT with 24h expiry + refresh token rotation	P0 — Critical
Security	All transport encrypted	HTTPS only (TLS 1.2+)	P0 — Critical
Security	Data at rest	PostgreSQL + S3 encryption at rest	P1 — High

Backup	Automated daily backups per tenant schema	5-year retention minimum	P1 — High
Browser Support	Chrome (primary), Edge, Firefox	Latest 2 versions each	P2 — Medium

18. Acceptance Criteria

The system is accepted when ALL of the following pass in the pilot tenant production environment:

18.1 Data Import

79. 2 years of historical Busy data (sales invoices, purchases, payment receipts) imports without data loss.
80. All unique SKUs, customers, and vendors auto-created on first import.
81. Subsequent imports do NOT overwrite is_focus_sku, season_tags, or msl_override.
82. Import summary shows: records total, imported, skipped, new masters created.
83. Failed rows are logged with row number and error reason, downloadable.

18.2 Forecasting & Inventory

84. DRR (4W, 13W, 52W, Recommended) correctly computed and match manual calculation for 5 test SKUs.
85. WOI correctly colour-coded Red/Amber/Green per spec thresholds.
86. System-suggested MSL uses formula: DRR_recommended x lead_time_days x 1.2.
87. Target stock quantity and suggested order quantity correctly computed using inventory_target_weeks setting. Verified at default (12w), then re-verified after changing setting to 8w.
88. WOI colour thresholds update immediately when woi_red_threshold / woi_amber_threshold are changed in Settings → Inventory Targets.
89. Forecasting recomputation completes in < 30 seconds for tenant's full SKU set.

18.3 Seasonal Forecasting

90. A Season Tag created for a SKU persists through 3 subsequent Busy imports.
91. For a tagged SKU with 1+ year of history, Seasonal DRR and Uplift % are shown in the forecast.
92. Pre-Season Order Alert correctly fires for a SKU whose season starts within 20 weeks.
93. Latest Order Date = season start date minus configured lead time days.
94. Stock Gap correctly computed as MAX(0, total_seasonal_demand - current_stock).

18.4 Profitability

95. Cost decoding formula saves and test-preview works correctly for 3 sample encoded values.

96. Re-decoding triggered on formula save; all historical purchase costs updated.
97. Gross Margin % computed correctly: (selling_price - decoded_cost) / selling_price × 100.
98. Volume-Profit Divergence correctly identifies SKUs in top-20 by volume but below median margin.
99. Decoded cost NOT visible to tenant_user role in any screen or report.

18.5 Customer Outstanding

100. Settings page allows switching between 'Direct Upload' and 'Computed' methods. Switching takes effect immediately.
101. METHOD A TEST: Set method = 'direct_upload'. Upload a Busy ledger file with 5 debit rows and 3 credit rows for a test customer. System computes correct net outstanding and assigns to correct ageing buckets (oldest-first allocation). Verified manually.
102. METHOD B TEST: Set method = 'computed'. Import 5 sales invoices for a test customer. Import partial payments against 3 of those invoices. System correctly computes invoice-level outstanding, customer total, and ageing buckets by invoice_date.
103. METHOD B: Invoice detail drill-down shows all unpaid invoices with correct Balance Due per invoice.
104. METHOD B: Overpayment (payments > invoice total) shows as credit balance, not negative bucket amount.
105. Both methods: report refreshes automatically after each relevant import with no manual action.
106. Both methods: WhatsApp button opens correct wa.me link with pre-filled {outstanding_amount} and {due_date} from oldest unpaid item.
107. Both methods: Export to Excel and PDF produces correct output with all ageing columns.
108. METHOD A upload zone is hidden when method = 'computed'. METHOD B upload zone is hidden when method = 'direct_upload'.

109. WhatsApp button opens correct number and pre-filled template message.
110. Export to Excel and PDF both work and include correct data.

18.6 Super Admin

111. New tenant provisioned in < 60 seconds: schema created, admin user created, onboarding email sent.
112. Tenant Admin receives onboarding email with working temporary credentials.
113. Suspended tenant cannot log in; reactivated tenant can.
114. All Super Admin actions appear in audit log with correct actor, action, timestamp.
115. No tenant user can access any /admin/* route (returns 403).

18.8 Branch & Location Management

116. Tenant provisioning auto-creates a Home Branch. The Home Branch cannot be deleted or deactivated.
117. AUTO-DETECTION TEST: Upload a Busy sales file containing a 'Sale From' column with values 'Store' and 'Warehouse'. System must: (1) detect the column, (2) surface a one-time

confirmation prompt, (3) on confirmation, auto-create two branches with source_label='Store' and source_label='Warehouse', (4) tag every row to the correct branch. No manual branch setup should be required.

118. REPEAT IMPORT TEST: Upload a second sales file of the same format. System must route rows to existing branches automatically using the saved branch_column_maps record — no prompt shown.
119. RENAME TEST: Rename the 'Warehouse' branch to 'Main Godown'. Upload a third file with Sale From = 'Warehouse'. System must still match to the renamed branch via source_label, not branch_name.
120. FALLBACK TEST: Upload a file with no Sale From column. The branch_id selected at upload time is applied to all rows.
121. BLANK SALE FROM TEST: Upload a file where some rows have a blank Sale From value. Those rows must be assigned to the Home Branch and flagged as warnings in the import error log.
122. MANUAL BRANCH TEST: Create a branch manually named 'Godown 2'. Upload a file with Sale From = 'Godown 2'. System matches it to the manual branch and populates its source_label.
123. Effective stock formula correctly accounts for: snapshot + purchases − sales + transfers_in − transfers_out, verified against manually computed figures for 3 test SKUs across 2 branches.
124. WOI and DRR are computed independently per branch and as a consolidated figure; both match manual calculations.
125. A stock transfer from Branch A to Branch B correctly reduces Branch A stock and increases Branch B stock in all reports.
126. Cross-Branch Comparison — all four tabs (Inventory, Sales, Profitability, Top SKUs) show correct data independently verified against the sales and purchases tables.
127. Dashboard Widget 18 (Cross-Branch Stock Alert) fires and links to Inventory tab pre-filtered.
128. Dashboard Widget 19 (Branch Sales Snapshot) shows correct revenue per branch; hidden when only one branch exists.
129. All four comparison tabs export to Excel with correct branch columns, period label, and data.

130. All reports (inventory, PO, profitability, seasonal) correctly filter by branch when a branch is selected.
131. Multi-branch import wizard: uploading a combined file and mapping a column to branches correctly splits records into the right branch_ids.

132. Attempting to access another tenant's data via manipulated JWT tenant_id returns 403.
133. All pages and API endpoints return 401 for unauthenticated requests.
134. Decoded purchase costs return 403 for tenant_user role.

— End of Document — IIS Dev SRS v2.0 — Internal Use Only —